

Contents

PART 4 — DIMENSIONALITY REDUCTION	3
Principal Component Analysis	7
1. Introduction	8
2. Why This Algorithm Was Developed	9
3. Real-life Motivation	9
4. Intuition	9
5. Working Principle	10
6. Mathematical Backend	11
7. Formula	12
8. Formula Derivation	12
9. Meaning of Every Symbol	13
10. Step-by-Step Formula Explanation	13
11. Numerical Example	14
12. Visualization	14
13. Hyperparameters & Attributes	15
14. Scikit-learn Import	18
15. Python Example	18
16. Transformation Process	19
17. Advantages	19
18. Disadvantages	20
19. Applications	20
20. Comparison with Previous Algorithm	21
21. Common Mistakes	21
22. Interview Questions	22
23. Summary	23
24. Complete Mathematical Implementation (Full Manual Walkthrough)	24
t-SNE (t-Distributed Stochastic Neighbor Embedding)	29
1. Introduction	29
2. Why This Algorithm Was Developed	29

3. Real-life Motivation	30
4. Intuition	30
5. Working Principle	30
6. Mathematical Backend	31
7. Formula	31
8. Formula Derivation	32
9. Meaning of Every Symbol	32
10. Step-by-Step Formula Explanation	33
11. Numerical Example	34
12. Visualization	34
13. Hyperparameters	35
14. Scikit-learn Import	36
15. Python Example	36
16. Transformation Process	38
17. Advantages	38
18. Disadvantages	38
19. Applications	39
20. Comparison with Previous Algorithm (PCA)	39
21. Common Mistakes	40
22. Interview Questions	41
23. Summary	42
24. Complete Mathematical Implementation (Full Manual Walkthrough)	42

PART 4 — DIMENSIONALITY REDUCTION

ডাইমেনশনালিটি রিডাকশন কী? (What is Dimensionality Reduction?)

Dimensionality Reduction হলো এমন একটি কৌশলগুচ্ছ, যার মাধ্যমে একটি ডেটাসেটের Feature বা Column-এর সংখ্যা (অর্থাৎ Dimension) কমিয়ে আনা হয়, কিন্তু ডেটার মধ্যে থাকা গুরুত্বপূর্ণ তথ্য (Information) এবং প্যাটার্ন যতটা সম্ভব অক্ষুণ্ণ রাখা হয়। প্রতিটি Feature-কে একটি axis বা মাত্রা হিসেবে চিন্তা করলে, ১০০টি Feature-যুক্ত একটি ডেটাসেট আসলে ১০০-মাত্রিক (100-dimensional) স্পেসে অবস্থিত একগুচ্ছ পয়েন্ট। মানুষ সর্বোচ্চ ৩টি মাত্রা (৩D) পর্যন্ত সরাসরি চোখে দেখতে ও বুঝতে পারে; এর বেশি মাত্রার ডেটা কল্পনা করা বা বিশ্লেষণ করা কার্যত অসম্ভব হয়ে ওঠে। Dimensionality Reduction এই সমস্যার সমাধান দেয় — উচ্চ-মাত্রিক ডেটাকে নিম্ন-মাত্রিক (সাধারণত ২D বা ৩D) স্পেসে রূপান্তর করে, যাতে ডেটা বিশ্লেষণ, ভিজুয়ালাইজেশন এবং পরবর্তী মডেলিং সহজ হয়।

উচ্চ-মাত্রিক ডেটা বিশ্লেষণ করা কেন কঠিন?

যত বেশি Feature যোগ হয়, ডেটা পয়েন্টগুলোর মধ্যকার দূরত্ব (Distance) হিসাব করা তত বেশি জটিল এবং কম অর্থবহ হয়ে পড়ে। উচ্চ-মাত্রিক স্পেসে ডেটা পয়েন্টগুলো একে অপর থেকে সমান দূরত্বে অবস্থান করতে শুরু করে (Distance Concentration), ফলে Nearest Neighbor-ভিত্তিক অ্যালগরিদম এবং Distance-ভিত্তিক মডেল (যেমন KNN, K-Means) কার্যকারিতা হারায়। একই সাথে, প্রতিটি নতুন Feature মডেলের জন্য নতুন প্যারামিটার যোগ করে, যা ট্রেনিং সময় এবং প্রয়োজনীয় ডেটার পরিমাণ বহুগুণ বাড়িয়ে দেয়।

Curse of Dimensionality (মাত্রার অভিশাপ)

Curse of Dimensionality এমন একটি ঘটনার নাম, যেখানে Feature বা মাত্রার সংখ্যা বাড়ার সাথে সাথে ডেটার Volume সূচকীয় (Exponentially) হারে বৃদ্ধি পায়, কিন্তু প্রকৃত ডেটা পয়েন্টের সংখ্যা একই থাকায় ডেটা ক্রমশ Sparse বা বিরল হয়ে যায়। ধরা যাক, ১টি Feature-এর জন্য একটি সরলরেখাকে ১০টি ভাগে ভাগ করলে ১০টি অংশ পাওয়া যায়; কিন্তু ২টি Feature হলে একই রেজোলিউশনে ১০০টি বর্গকোষ (10×10) প্রয়োজন হয়, ৩টি Feature হলে ১০০০টি ঘনকোষ ($10 \times 10 \times 10$) প্রয়োজন হয়। অর্থাৎ প্রতিটি নতুন মাত্রা যোগ হওয়ার সাথে সাথে সমান ঘনত্বে ডেটা

পূর্ণ করতে সূচকীয় হারে বেশি Sample প্রয়োজন হয় — বাস্তবে যা প্রায় কখনোই সম্ভব হয় না। ফলে মডেল প্রশিক্ষণের জন্য পর্যাপ্ত ঘনত্বের ডেটা পাওয়া যায় না এবং Overfitting-এর ঝুঁকি বহুগুণ বেড়ে যায়।

Feature Compression এবং Information Preservation

Dimensionality Reduction-এর মূল কৌশল হলো Feature Compression — অর্থাৎ একাধিক পারস্পরিক-সম্পর্কযুক্ত (Correlated) Feature-কে কম সংখ্যক নতুন Feature-এ সংকুচিত করা, যেখানে প্রতিটি নতুন Feature মূল ডেটার একাধিক পুরনো Feature-এর তথ্য একত্রে ধারণ করে। এই প্রক্রিয়ায় লক্ষ্য থাকে Information Preservation — অর্থাৎ যতটা সম্ভব কম Feature ব্যবহার করে ডেটার Variance বা Structure-এর সর্বোচ্চ অংশ ধরে রাখা, যাতে মূল ডেটার গুরুত্বপূর্ণ প্যাটার্ন হারিয়ে না যায়।

Visualization, Noise Reduction এবং Computational Efficiency

- Visualization: মানুষ ২D বা ৩D পর্যন্ত ডেটা সরাসরি চোখে দেখে বুঝতে পারে; উচ্চ-মাত্রিক ডেটাকে ২D/৩D-তে নামিয়ে আনলে Cluster, Outlier বা প্যাটার্ন সহজেই চোখে ধরা পড়ে।
- Noise Reduction: প্রায়ই ডেটার মধ্যে থাকা Low-Variance Feature বা Dimension মূলত Noise বহন করে; সেগুলো বাদ দিলে ডেটা পরিষ্কার (Clean) হয় এবং মডেলের Generalization ক্ষমতা বাড়ে।
- Computational Efficiency: কম Feature নিয়ে কাজ করলে মেমোরি ব্যবহার, ট্রেনিং সময় এবং প্রেডিকশন সময় — সবকিছুই উল্লেখযোগ্যভাবে কমে যায়, বিশেষ করে Distance-ভিত্তিক বা Matrix-ভিত্তিক অ্যালগরিদমে।

Feature Selection বনাম Feature Extraction

Dimensionality Reduction করার দুইটি মৌলিক দর্শন বা পদ্ধতি রয়েছে — Feature Selection এবং Feature Extraction। দুটোই মাত্রা কমায়, কিন্তু সম্পূর্ণ ভিন্ন পথে।

Feature Selection (ফিচার নির্বাচন)

Feature Selection-এ মূল ডেটাসেটের সবগুলো Feature-এর মধ্য থেকে সবচেয়ে গুরুত্বপূর্ণ কিছু Feature বেছে নেওয়া হয় এবং বাকিগুলো সম্পূর্ণভাবে বাদ দেওয়া হয়। এখানে নির্বাচিত Feature-গুলো তাদের আদি (Original) অর্থ ও একক (Unit) ধরে রাখে — যেমন যদি 'Age' এবং 'Salary' নির্বাচিত হয়, তাহলে সেগুলো এখনো 'বয়স' এবং 'বেতন' হিসেবেই থেকে যায়, নতুন কোনো রূপান্তরিত মান তৈরি হয় না। Filter Method (Correlation, Chi-Square), Wrapper Method (Recursive Feature Elimination), এবং Embedded Method (Lasso-এর মতো মডেলের অন্তর্নিহিত Feature গুরুত্ব) — এই তিনটি প্রধান কৌশলে Feature Selection করা হয়।

Feature Extraction (ফিচার নিষ্কাশন)

Feature Extraction-এ মূল Feature-গুলোকে গাণিতিকভাবে একত্রিত বা রূপান্তর করে সম্পূর্ণ নতুন কিছু Feature তৈরি করা হয়, যেগুলোর সংখ্যা মূল Feature-এর চেয়ে কম কিন্তু প্রতিটি নতুন Feature আসলে একাধিক পুরনো Feature-এর তথ্যের একটি সমন্বিত রূপ (Linear বা Non-linear Combination)। এই নতুন Feature-গুলোর কোনো সরাসরি বাস্তব-জগতের অর্থ থাকে না — যেমন PCA-এর মাধ্যমে তৈরি হওয়া 'Principal Component 1' আসলে একাধিক আদি Feature-এর একটি ওয়েটেড কম্বিনেশন, যার নিজস্ব কোনো একক (যেমন টাকা বা বছর) থাকে না। PCA এবং t-SNE — উভয়েই Feature Extraction-ভিত্তিক অ্যালগরিদম।

বিষয়	Feature Selection	Feature Extraction
মূল কৌশল	বিদ্যমান Feature থেকে সেরাগুলো বেছে নেওয়া	Feature-গুলোকে একত্রিত করে নতুন Feature তৈরি
আদি অর্থ	বজায় থাকে (Interpretable)	হারিয়ে যায় (Less Interpretable)
উদাহরণ অ্যালগরিদম	Chi-Square, RFE, Lasso	PCA, t-SNE, LDA, Autoencoder
তথ্য ক্ষয়	বাদ দেওয়া Feature-এর তথ্য সম্পূর্ণ হারায়	সব Feature-এর আংশিক তথ্য নতুন Feature-এ মিশে থাকে
ব্যবহারের ক্ষেত্র	যখন Feature-এর ব্যাখ্যা গুরুত্বপূর্ণ	যখন Compression বা Visualization মুখ্য উদ্দেশ্য

এই Part-এর অধ্যায়সমূহ (Chapters in this Part)

- Chapter 17 — Principal Component Analysis (PCA): একটি Linear, Unsupervised Feature Extraction পদ্ধতি, যা ডেটার সর্বোচ্চ Variance বজায় রেখে নতুন Orthogonal অক্ষ (Principal Components) ডেটা প্রজেক্ট করে।
- Chapter 18 — t-SNE (t-Distributed Stochastic Neighbor Embedding): একটি Non-linear, Unsupervised Visualization পদ্ধতি, যা উচ্চ-মাত্রিক স্পেসের স্থানীয় (Local) প্রতিবেশী সম্পর্ক নিম্ন-মাত্রিক স্পেসে যতটা সম্ভব অক্ষুণ্ণ রেখে ডেটা ভিজুয়ালাইজ করে।

কেন PCA আগে এবং t-SNE পরে?

এই Part-এ PCA-কে t-SNE-এর আগে রাখা হয়েছে কারণ PCA একটি Linear এবং তুলনামূলক সরল গাণিতিক ভিত্তির (Eigen Decomposition) উপর দাঁড়িয়ে আছে, যা Dimensionality Reduction-এর মূল ধারণা — Variance সংরক্ষণ, Projection, এবং Information Compression — সহজভাবে বুঝতে সাহায্য করে। এই ভিত্তি বোঝার পরই t-SNE-এর মতো Non-linear, Probability-ভিত্তিক এবং তুলনামূলক জটিল (Gradient Descent, KL Divergence, Student t-Distribution) কৌশল বোঝা সহজ হয়ে যায়। এছাড়া বাস্তব জগতে PCA প্রায়ই t-SNE-এর একটি প্রাথমিক ধাপ (Preprocessing Step) হিসেবে ব্যবহৃত হয়, তাই যৌক্তিক ক্রম হিসেবেও PCA আগে আসা স্বাভাবিক।

কখন PCA প্রাধান্য পায়

- যখন উদ্দেশ্য শুধু Visualization নয়, বরং ডেটাকে পরবর্তী কোনো Machine Learning মডেলের জন্য প্রস্তুত করা (Preprocessing/Feature Engineering)।
- যখন Feature-গুলোর মধ্যে মূলত Linear সম্পর্ক (Correlation) বিদ্যমান।
- যখন দ্রুতগতির, Deterministic এবং নতুন ডেটাতে সহজে প্রয়োগযোগ্য (transform()) একটি সমাধান প্রয়োজন।
- যখন Interpretability গুরুত্বপূর্ণ — কোন Feature কতটা Variance ব্যাখ্যা করছে তা জানা দরকার।

কখন t-SNE প্রাধান্য পায়

- যখন মূল উদ্দেশ্য উচ্চ-মাত্রিক ডেটার Cluster বা Local Structure মানুষের চোখে দেখানো (Pure Visualization)।
- যখন ডেটার মধ্যে সম্পর্ক Non-linear এবং জটিল, যা PCA-এর সরলরৈখিক প্রজেকশনে ধরা পড়ে না।
- যখন গতি বা নতুন ডেটাতে সরাসরি প্রয়োগযোগ্যতা মুখ্য বিষয় নয়, বরং ভিজুয়াল স্বচ্ছতা বেশি গুরুত্বপূর্ণ।

এই Part কীভাবে Ensemble Learning-এর জন্য প্রস্তুত করে

Dimensionality Reduction শেখার মাধ্যমে পাঠক বুঝতে পারবেন কীভাবে অপ্রয়োজনীয় বা Redundant তথ্য বাদ দিয়ে ডেটাকে আরও পরিষ্কার ও কার্যকরভাবে উপস্থাপন করা যায় — এই একই চিন্তাধারা পরবর্তী Ensemble Learning অংশে অত্যন্ত গুরুত্বপূর্ণ হয়ে ওঠে, যেখানে একাধিক দুর্বল মডেলের (Weak Learner) সিদ্ধান্তকে কার্যকরভাবে একত্রিত করে একটি শক্তিশালী মডেল তৈরি করা হয়। Dimensionality Reduction-এ যেমন 'কোন তথ্য গুরুত্বপূর্ণ, কোনটি Noise' তা আলাদা করতে শেখা যায়, তেমনি Ensemble Learning-এও 'কোন মডেলের সিদ্ধান্ত বেশি নির্ভরযোগ্য' তা মূল্যায়ন করার একই ধরনের বিশ্লেষণী দৃষ্টিভঙ্গি কাজে লাগে। এছাড়া, বাস্তব প্রজেক্টে প্রায়ই PCA দিয়ে Feature সংকুচিত করার পর সেই Reduced Feature-এর উপর Ensemble মডেল (Random Forest, Gradient Boosting) প্রশিক্ষণ দেওয়া একটি প্রচলিত Best Practice।

Principal Component Analysis

নিচের অংশে PCA-কে একটি সম্পূর্ণ, একক ও ধারাবাহিক কাঠামোতে (Section 1 থেকে 24) উপস্থাপন করা হলো, যেখানে ধারণাগত আলোচনা, সম্পূর্ণ গাণিতিক ডেরিভেশন, বাস্তব সংখ্যাসূচক উদাহরণ, Python কোড, Scikit-learn-এর প্রতিটি হাইপারপ্যারামিটার/অ্যাট্রিবিউট এবং ইন্টারভিউ প্রশ্ন — সবকিছু একত্রে, একবার করে বিস্তারিতভাবে আলোচনা করা হয়েছে।

1. Introduction

Principal Component Analysis (PCA) হলো একটি Linear, Unsupervised Feature Extraction অ্যালগরিদম, যা ডেটাসেটের মূল তথ্য বা বৈশিষ্ট্য অক্ষুণ্ণ রেখে Feature-এর সংখ্যা (Dimension) কমিয়ে আনে। এটি মূল Feature-গুলোকে সরাসরি বাদ দেয় না — বরং সবগুলো Feature-এর একটি Linear Combination নিয়ে সম্পূর্ণ নতুন, পারস্পরিক Uncorrelated কিছু Feature তৈরি করে, যাদের Principal Component (PC) বলা হয়। প্রথম Component (PC1) ডেটার সর্বোচ্চ Variance ধারণ করে, দ্বিতীয়টি (PC2) তার পরের সর্বোচ্চ Variance ধারণ করে, এভাবে চলতে থাকে। (Feature Selection ও Feature Extraction-এর মধ্যে মৌলিক পার্থক্য Part 4-এর ভূমিকায় বিস্তারিত আলোচনা করা হয়েছে — PCA একটি Feature Extraction পদ্ধতি, কোনো Feature Selection পদ্ধতি নয়।)

Curse of Dimensionality — জ্যামিতিক দৃষ্টিকোণ থেকে

PCA কেন প্রয়োজন তা বোঝার জন্য দেখা যাক ডাইমেনশন বাড়ার সাথে সাথে ডেটা-স্পেসের জ্যামিতিক আকৃতি কীভাবে বদলায়। ধরা যাক, একই ১০টি ডেটা পয়েন্ট নিয়ে কাজ করা হচ্ছে। ১টি Feature (1D) থাকলে এগুলো একটি ১০-ইউনিট দৈর্ঘ্যের রেখায় বসে, গড়ে ১ ইউনিট দূরত্বে। ২টি Feature (2D) হলে একই রেজোলিউশনে ১০০ বর্গ-ইউনিটের একটি সমতল লাগে (10×10), অধিকাংশ ঘর ফাঁকা থেকে যায়। ৩টি Feature (3D) হলে ১০০০ ঘন-ইউনিট প্রয়োজন হয় (10×10×10), এবং স্পেসের প্রায় ৯৯% ফাঁকা থাকে। ১০০টি Feature হলে প্রয়োজনীয় স্পেসের আয়তন কার্যত অসীমের কাছাকাছি চলে যায় — এই ঘটনাকেই Curse of Dimensionality বলা হয়।

এর ফলে চারটি নির্দিষ্ট সমস্যা তৈরি হয়: (১) Data Sparsity — স্পেসের আয়তন সূচকীয় হারে বাড়লেও Sample সংখ্যা একই থাকায় পুরো স্পেস ফাঁকা হয়ে যায়; (২) Distance Concentration Effect — উচ্চ-মাত্রায় সবচেয়ে কাছের ও সবচেয়ে দূরের পয়েন্টের দূরত্বের ব্যবধান প্রায় শূন্যের কাছাকাছি চলে আসে, ফলে KNN বা SVM-এর মতো Distance-ভিত্তিক অ্যালগরিদম আসল প্রতিবেশী চিনতে ব্যর্থ হয়; (৩) Hughes Phenomenon — একটি মডেলকে নিখুঁতভাবে ট্রেন করতে Feature সংখ্যার তুলনায় সূচকীয় হারে বেশি Sample প্রয়োজন হয়, যা বাস্তবে প্রায়ই অসম্ভব, ফলে Overfitting ঘটে; (৪) Computational Explosion — Feature বাড়ার সাথে সাথে

প্রতিটি দূরত্ব ও ম্যাট্রিক্স-গণনার খরচ বহুগুণ বেড়ে যায়। PCA এই চারটি সমস্যাই সমাধান করে — ডেটার Variance অক্ষুণ্ণ রেখে কম সংখ্যক Component-এ তথ্য ঘনীভূত করার মাধ্যমে।

2. Why This Algorithm Was Developed

বিংশ শতাব্দীর শুরুতে পরিসংখ্যানবিদরা এমন একটি সমস্যার মুখোমুখি হচ্ছিলেন — একাধিক Correlated (পারস্পরিক সম্পর্কযুক্ত) পরিমাপ (যেমন উচ্চতা, ওজন, বুকের মাপ) একসাথে বিশ্লেষণ করতে গিয়ে অতিরিক্ত জটিলতা তৈরি হচ্ছিল, কারণ প্রকৃত স্বাধীন তথ্যের (Independent Information) পরিমাণ প্রকৃত Feature সংখ্যার চেয়ে অনেক কম ছিল। Karl Pearson ১৯০১ সালে এই সমস্যার সমাধান হিসেবে এমন একটি পদ্ধতি প্রস্তাব করেন যা বহু Correlated ভেরিয়েবলকে সংক্ষিপ্ত করে অল্প কিছু Uncorrelated ভেরিয়েবলে রূপান্তর করতে পারে। ১৯৩৩ সালে Harold Hotelling এটিকে গাণিতিকভাবে প্রতিষ্ঠিত করেন, যা PCA-কে Machine Learning-এর অন্যতম প্রাচীন গাণিতিক ভিত্তির অ্যালগরিদম করে তোলে।

3. Real-life Motivation

কল্পনা করুন, একটি বিশ্ববিদ্যালয়ের শিক্ষার্থীদের ১০টি বিষয়ের নম্বর রেকর্ড করা আছে। বাস্তবে দেখা যায়, যারা Math-এ ভালো করে তারা প্রায়ই Physics এবং Chemistry-তেও ভালো করে। ফলে ১০টি আলাদা Feature আসলে হয়তো মাত্র ২টি প্রকৃত অন্তর্নিহিত বৈশিষ্ট্যকে (যেমন 'বিশ্লেষণী দক্ষতা' এবং 'ভাষাগত দক্ষতা') প্রতিফলিত করছে। PCA এই লুকানো কাঠামো খুঁজে বের করে ১০টি Feature-কে মাত্র ২টি অর্থবহ Component-এ সংকুচিত করতে পারে। শিল্পক্ষেত্রেও PCA-এর ব্যাপক ব্যবহার আছে — মুখাবয়ব চেনার (Face Recognition) Eigenfaces পদ্ধতি PCA-এর উপর ভিত্তি করে তৈরি, জিনোমিক ডেটায় হাজার হাজার জিন এক্সপ্রেশন থেকে মূল প্যাটার্ন খুঁজতে, এবং আর্থিক বাজারে বহু স্টকের মূল্য-পরিবর্তনের মধ্যে সাধারণ ঝুঁকি-উপাদান (Risk Factor) চিহ্নিত করতে PCA নিয়মিত ব্যবহৃত হয়।

4. Intuition

PCA-কে বোঝার সবচেয়ে সহজ উপায় হলো একটি প্রলম্বিত (Elongated) মেঘের মতো ছড়িয়ে থাকা ডেটা ক্লাউড কল্পনা করা। এই ডেটা ক্লাউডের উপর যে সরলরেখাটি ডেটার দৈর্ঘ্য বরাবর

সবচেয়ে বেশি বিস্তৃতি (Spread) ধরে রাখে, সেটিই হলো PC1। এরপর PC1-এর সাথে লম্বভাবে (Perpendicular) আরেকটি রেখা আঁকা হয়, যা অবশিষ্ট সর্বোচ্চ Variance ধরে রাখে — সেটিই PC2। গুরুত্বপূর্ণ বিষয় হলো, PCA কোনো Feature-কে সরাসরি বাদ দেয় না; বরং সবগুলো মূল Feature-কে গাণিতিকভাবে একত্রিত করে নতুন অক্ষ তৈরি করে, এবং যেসব নতুন অক্ষ অত্যন্ত কম Variance ধারণ করে সেগুলোকে বাদ দেওয়া হয় — কারণ কম-Variance দিকগুলো সাধারণত Noise বহন করে।

একটি বাস্তব সিমুলেশন — গাড়ির ইঞ্জিন সাইজ ও হর্সপাওয়ার

ধরা যাক, ৫০০টি গাড়ির ডেটাসেটে দুটি Feature ট্র্যাক করা হচ্ছে — Engine Size এবং Horsepower। এই দুটিকে একটি 2D গ্রিডে প্লট করলে ডেটা ক্লাউডটি প্রথমে গ্রিডের যেকোনো একপাশে অবস্থান করে (গড় মান শূন্য নয়)। PCA প্রথমে ৫০০টি গাড়ির গড় Engine Size ও গড় Horsepower হিসাব করে পুরো ক্লাউডটিকে এমনভাবে সরিয়ে নেয় যাতে এই গড় বিন্দুটি গ্রাফের ঠিক কেন্দ্রে (0,0) বসে — এটিই Mean Centering। এরপর মূলবিন্দু দিয়ে যাওয়া একটি সরলরেখাকে ততক্ষণ ঘুরানো হয় যতক্ষণ না এটি ডেটা ক্লাউডের সবচেয়ে বিস্তৃত দিকের সাথে নিখুঁতভাবে মিলে যায় — যেহেতু Engine Size বাড়লে Horsepower-ও বাড়ে, তাই এই বেস্ট-ফিট লাইনটি কোণাকুণিভাবে ওপরের দিকে হলে থাকে, এবং এটিই PC1। এরপর PC1-এর সাথে ৯০° কোণে আরেকটি রেখা (PC2) আঁকা হয়, যা সেই ছোটখাটো বৈচিত্র্য ধরে রাখে যা PC1 মিস করেছিল (যেমন কোনো গাড়ির Horsepower তার Engine Size অনুপাতে সামান্য কম/বেশি হওয়া)। সবশেষে পুরো গ্রাফটিকে এমনভাবে ঘুরিয়ে দেওয়া হয় যাতে PC1 আনুভূমিক ও PC2 উল্লম্ব অক্ষে পরিণত হয়; যদি দেখা যায় PC1 একাই ৯৫% Variance ধারণ করেছে, তাহলে PC2 বাদ দিয়ে দিলেও কোনো বড় তথ্যের ক্ষতি হয় না — ২D ডেটাটি নিরাপদে একটি ১D রেখায় সংকুচিত হয়ে যায়।

5. Working Principle

- ধাপ ১ — Standardization/Mean Centering: প্রতিটি Feature থেকে তার নিজস্ব Mean বিয়োগ করে ডেটাকে কেন্দ্রীভূত করা হয় এবং Variance ১-এ স্কেল করা হয়, যাতে বড় স্কেলের Feature (যেমন বেতন) ছোট স্কেলের Feature-কে (যেমন বয়স) অন্যায়ভাবে প্রভাবিত না করে।

- ধাপ ২ — Covariance Matrix Construction: কেন্দ্রীভূত ডেটা থেকে একটি Covariance Matrix তৈরি করা হয়। এর ডায়াগোনাল উপাদানগুলো প্রতিটি Feature-এর নিজস্ব Variance নির্দেশ করে, আর অফ-ডায়াগোনাল উপাদানগুলো দুই ভিন্ন Feature-এর মধ্যে Covariance দেখায় — অফ-ডায়াগোনাল মান বেশি হওয়া মানে দুই Feature Redundant, প্রায় একই তথ্য বহন করছে।
- ধাপ ৩ — Eigen Decomposition: Covariance Matrix থেকে Eigenvalue ও Eigenvector গণনা করা হয়। প্রথম Eigenvector ডেটা ক্লাউডের সর্বোচ্চ Variance-এর দিক বরাবর নির্দেশ করে এবং PC1 হিসেবে চিহ্নিত হয়; সংশ্লিষ্ট Eigenvalue বলে দেয় সেই দিকে ঠিক কতটুকু Variance ধরে রাখা গেছে।
- ধাপ ৪ — Orthogonality Constraint: PC2 এমনভাবে নির্ধারণ করা হয় যাতে এটি PC1-এর সাথে নিখুঁতভাবে লম্ব (৯০°) হয় এবং অবশিষ্ট সর্বোচ্চ Variance ধারণ করে। এই প্রক্রিয়া PC3, PC4 থেকে PCn পর্যন্ত ধারাবাহিকভাবে চলতে থাকে — প্রতিটি নতুন Component আগের সবগুলোর সাথে Orthogonal থাকায় Multicollinearity সম্পূর্ণভাবে দূর হয়।
- ধাপ ৫ — Projection/Transformation: নির্বাচিত Eigenvector দিয়ে তৈরি Projection Matrix দিয়ে মূল কেন্দ্রীভূত ডেটাকে নতুন অক্ষে প্রজেক্ট করা হয়। কম Eigenvalue-বিশিষ্ট দুর্বল Component বাদ দিয়ে একটি সংকুচিত, পরিচ্ছন্ন ডেটাসেট তৈরি হয়, যা মেশিন লার্নিং মডেলের জন্য প্রস্তুত।

6. Mathematical Backend

PCA-এর গাণিতিক ভিত্তি Linear Algebra-এর দুটি স্তরের উপর দাঁড়িয়ে আছে — Covariance Matrix এবং Eigen Decomposition। যেহেতু Covariance Matrix সবসময় Symmetric এবং Positive Semi-Definite, তাই এর Eigenvalue সবসময় বাস্তব সংখ্যা ও অ-ঋণাত্মক, এবং Eigenvector-গুলো একে অপরের সাথে Orthogonal হয় — এই বৈশিষ্ট্য PCA-কে গাণিতিকভাবে সুসংজ্ঞায়িত করে তোলে। মূল লক্ষ্য এমন একটি একক ভেক্টর w ($\|w\|=1$) খুঁজে বের করা, যেখানে ডেটাকে w -এর উপর প্রজেক্ট করলে প্রাপ্ত মানগুলোর Variance সর্বোচ্চ হয়। Lagrange

Multiplier দিয়ে এই Constrained Optimization সমাধান করলে দেখা যায় — সর্বোচ্চ Variance দেয় এমন w অবশ্যই Covariance Matrix-এর একটি Eigenvector হতে হবে।

7. Formula

$$\text{Covariance Matrix: } \Sigma = [1/(n-1)] \cdot X^{cT} \cdot X^c$$

$$\text{Eigen Equation: } \Sigma \cdot v = \lambda \cdot v$$

$$\text{Projection: } Z = X^c \cdot V$$

যেখানে X^c হলো Mean-Centered ডেটা ম্যাট্রিক্স, Σ হলো Covariance Matrix, v হলো Eigenvector, λ হলো সংশ্লিষ্ট Eigenvalue, V হলো নির্বাচিত Eigenvector-গুলোর কলাম দিয়ে তৈরি Projection Matrix, এবং Z হলো Reduced-Dimension ফলাফল।

8. Formula Derivation

ধরা যাক, ডেটা ম্যাট্রিক্স X -এর n সংখ্যক Sample এবং p সংখ্যক Feature আছে। প্রথমে $X^c = X - X$ দিয়ে প্রতিটি Feature-কে শূন্য-কেন্দ্রিক করা হয়। ডেটাকে একটি একক Unit Vector w -এর উপর প্রজেক্ট করলে $z = X^c \cdot w$ পাওয়া যায়, এবং এর Variance:

$$\text{Var}(z) = (1/(n-1)) \cdot (X^c w)^T (X^c w) = w^T \cdot [(1/(n-1)) \cdot X^{cT} X^c] \cdot w = w^T \Sigma w$$

লক্ষ্য হলো এই $w^T \Sigma w$ সর্বোচ্চ করা, শর্তসাপেক্ষে $w^T w = 1$ । Lagrange Multiplier প্রয়োগ করে:

$$L(w, \lambda) = w^T \Sigma w - \lambda(w^T w - 1)$$

w -এর সাপেক্ষে ডেরিভেটিভ নিয়ে শূন্যের সমান বসালে:

$$\partial L / \partial w = 2 \Sigma w - 2 \lambda w = 0 \Rightarrow \Sigma w = \lambda w$$

এটিই ক্লাসিক্যাল Eigenvalue সমীকরণ — প্রমাণ করে যে Variance সর্বোচ্চ করে এমন w অবশ্যই Σ -এর একটি Eigenvector হতে হবে, এবং সেই দিকে Variance-এর মান ঠিক সংশ্লিষ্ট λ -এর সমান ($w^T \Sigma w = w^T \lambda w = \lambda$)। সবচেয়ে বড় Eigenvalue-যুক্ত Eigenvector হলো PC1, তার পরেরটি (PC1-এর সাথে Orthogonal) PC2, ইত্যাদি।

9. Meaning of Every Symbol

সিঙ্ঘল	অর্থ
X	মূল ডেটা ম্যাট্রিক্স (n samples $\times$ p features)
$\bar{X}$	প্রতিটি Feature-এর Mean
X^c	Mean-Centered ডেটা ম্যাট্রিক্স ($X - \bar{X}$)
Σ	Covariance Matrix ($p \times p$ Symmetric ম্যাট্রিক্স)
n	মোট Sample সংখ্যা
v বা w	Eigenvector — নতুন অক্ষের দিক নির্দেশক একক ভেক্টর
λ	Eigenvalue — সংশ্লিষ্ট অক্ষ বরাবর Variance-এর পরিমাণ
V	নির্বাচিত Eigenvector-গুলোর Projection Matrix
Z	Reduced-Dimension প্রজেক্টেড ডেটা

10. Step-by-Step Formula Explanation

- প্রথমে $X^c = X - \bar{X}$ হিসাব করে প্রতিটি Feature-কে Zero-Centered করা হয়।
- $\Sigma = X^{cT}X^c/(n-1)$ সূত্রে Covariance Matrix গণনা করা হয়।
- $\det(\Sigma - \lambda I) = 0$ (Characteristic Equation) সমাধান করে λ বের করা হয়, তারপর প্রতিটি λ -এর জন্য $(\Sigma - \lambda I)v = 0$ সমাধান করে v বের করা হয়।
- Eigenvalue-গুলোকে বৃহৎ থেকে ক্ষুদ্র ক্রমে সাজিয়ে প্রয়োজনীয় সংখ্যক ($n_components$) বৃহত্তম Eigenvector নিয়ে V তৈরি করা হয়।
- $Z = X^c \cdot V$ সূত্রে মূল ডেটাকে নতুন, কম-মাত্রিক অক্ষে প্রজেক্ট করা হয়।

11. Numerical Example

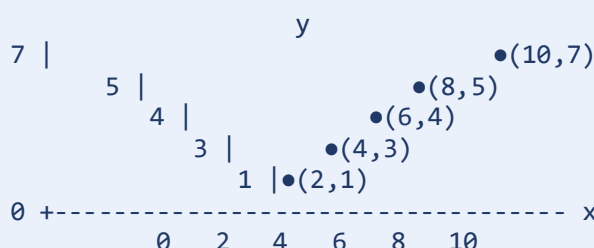
নিচের ৫টি Sample এবং ২টি Feature-যুক্ত ছোট ডেটাসেটটি ব্যবহার করে PCA-এর প্রতিটি ধাপ Section 24-এ সম্পূর্ণভাবে হাতে-কলমে দেখানো হয়েছে।

Sample	Feature 1 (x)	Feature 2 (y)
1	2	1
2	4	3
3	6	4
4	8	5
5	10	7

Mean বিয়োগ করার পর কেন্দ্রীভূত ডেটা পাওয়া যায় $(-4, -3), (-2, -1), (0, 0), (2, 1), (4, 3)$, যা থেকে Covariance Matrix $\Sigma = [[10, 7], [7, 5]]$ পাওয়া যায়। এর Eigenvalue দুটি হলো $\lambda_1 \approx 14.9330$ এবং $\lambda_2 \approx 0.0670$, এবং সংশ্লিষ্ট Eigenvector দুটি হলো $v_1 \approx (0.8174, 0.5760)$ এবং $v_2 \approx (0.5760, -0.8174)$ । যেহেতু λ_1 মোট Variance-এর প্রায় 99.55% ধারণ করে, শুধু PC1 রাখলেও প্রায় সম্পূর্ণ তথ্য সংরক্ষিত থাকে।

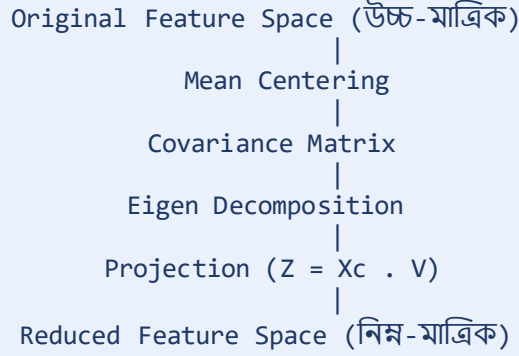
12. Visualization

মূল ফিচার স্পেস (Feature 1 vs Feature 2)



ডেটা প্রায় একটি সরলরেখা বরাবর বিস্তৃত – এই দিকটিই PC1

PCA রূপান্তর প্রক্রিয়া (Transformation Pipeline)



13. Hyperparameters & Attributes

`sklearn.decomposition.PCA` মডেলের ভেতরের কার্যপ্রক্রিয়া নিয়ন্ত্রণ করতে নিচের হাইপারপ্যারামিটারগুলো ব্যবহার করা হয়, গুরুত্বের ক্রমানুসারে।

n_components (Default: None)

PCA রূপান্তরের পর সর্বমোট কতটি Principal Component রাখা হবে তা নির্ধারণ করে — ডাইমেনশনালিটি রিডাকশনের জন্য বাস্তব প্রজেক্টে এটিই সবচেয়ে বেশি টিউন করা হয়। None রাখলে মূল Feature সংখ্যার সমান Component তৈরি হয়। কোনো পূর্ণসংখ্যা (যেমন 2 বা 3) দিলে শুধু সেই কয়টি শীর্ষ Component রাখা হয় (Visualization-এর জন্য উপযোগী)। শূন্য থেকে ১-এর মধ্যে একটি ভগ্নাংশ (যেমন 0.95) দিলে Cumulative Variance Threshold তত্ত্ব ব্যবহার করে স্বয়ংক্রিয়ভাবে ততগুলো Component বেছে নেওয়া হয় যা মূল ডেটার অন্তত ৯৫% Variance ধরে রাখে।

svd_solver (Default: 'auto')

Eigen Decomposition বা SVD গণনার নির্দিষ্ট গাণিতিক অ্যালগরিদম নির্বাচন করে। 'full' ছোট/মাঝারি ডেটাসেটে সম্পূর্ণ প্রথাগত SVD গণনা করে নিখুঁত ফলাফল দেয়; 'arpack' একটি Iterative পদ্ধতি, যখন খুব কম সংখ্যক Component ($n_components \ll n_features$) দরকার তখন দ্রুত কনভার্জ করে; 'randomized' বিশাল ডেটাসেটে (লাখ লাখ Row) একটি Truncated/Randomized SVD ব্যবহার করে দ্রুত আনুমানিক ফলাফল দেয়।

whiten (Default: False)

প্রতিটি Component-এর আউটপুটকে তার নিজস্ব Eigenvalue-এর বর্গমূল দিয়ে ভাগ করে, যাতে প্রতিটি Component-এর Variance ফিক্সড (১) হয়ে যায়। True করলে ডাউনস্ট্রিম মডেলের (Neural Network, SVM) Optimization আরও দ্রুত ও স্থিতিশীল হয়, কারণ প্রতিটি নতুন Feature একই স্কেলে থাকে।

random_state (Default: None)

svd_solver='randomized' বা 'arpack' ব্যবহারের সময় ফলাফল Reproducible রাখতে একটি Fixed পূর্ণসংখ্যা (যেমন 42) সেট করা হয়, যাতে কোড যতবারই চালানো হোক, একই Component ও একই Coordinate পাওয়া যায়।

copy (Default: True), tol (Default: 0.0), iterated_power (Default: 'auto'), n_oversamples (Default: 10), power_iteration_normalizer (Default: 'auto')

copy=False দিলে মেমোরি বাঁচাতে মূল ডেটার উপর সরাসরি In-Place রূপান্তর করা হয় (মূল ডেটা পরিবর্তিত হয়ে যায়)। tol হলো svd_solver='arpack'-এর Iteration থামানোর Convergence Tolerance — বড় ডেটাসেটে সামান্য বাড়িয়ে দিলে দ্রুত ট্রেনিং শেষ হয়। iterated_power শুধু svd_solver='randomized'-এর সাথে কাজ করে, Power Iteration-এর সংখ্যা নিয়ন্ত্রণ করে আরও নিখুঁত Eigenvector খুঁজতে সাহায্য করে (NLP-এর মতো অত্যন্ত Sparse ডেটাসেটে ম্যানুয়ালি বাড়ানো হয়)। n_oversamples শুধু 'randomized' মোডে অতিরিক্ত Oversample নিয়ে গাণিতিক স্থায়িত্ব বাড়ায় (মেডিকেল ইমেজের মতো সংবেদনশীল ডেটায় উপযোগী)। power_iteration_normalizer ('QR', 'LU', 'none') Power Iteration-এর প্রতিটি ধাপে Numerical Stability বজায় রাখে, সাধারণ প্রজেক্টে ডিফল্ট রাখাই যথেষ্ট।

কম্পোনেন্ট সিলেক্ট করার তাত্ত্বিক পদ্ধতি

কতগুলো Component রাখা উচিত তা নির্ধারণ করতে তিনটি তাত্ত্বিক পদ্ধতি ব্যবহার করা হয় — (১) Scree Plot: Component সংখ্যা বনাম Variance-এর একটি লেখচিত্র আঁকা হয়, যেখানে গ্রাফটি হঠাৎ সমতল হয়ে যাওয়ার বিন্দুকে (Elbow) সীমানা ধরা হয়; (২) Cumulative Variance Threshold: PC1 থেকে শুরু করে একে একে Variance যোগ করে যতগুলো Component-এ কাক্সিক্ষত শতাংশ (যেমন ৯৫%) পৌঁছায় ততগুলো রাখা হয়; (৩) Kaiser's Criterion: একটি

Standardized ডেটাসেটে প্রতিটি মূল Feature-এর গড় Eigenvalue হয় ১.০, তাই যে Component-এর Eigenvalue ১.০-এর কম, সেটিকে গড়পড়তা একটি Feature-এর সমান তথ্যও ধরে রাখতে না পারায় বাদ দেওয়া হয়।

মূল Attributes (মডেল fit করার পর)

Attribute	Shape/Type	অর্থ
components_	(n_components, n_features)	প্রতিটি Principal Component-এর দিক নির্দেশকারী Eigenvector, Variance অনুযায়ী সাজানো
explained_variance_ratio_	(n_components,)	প্রতিটি Component মোট তথ্যের কত শতাংশ ধরে রেখেছে (যোগফল সবসময় 1.0)
explained_variance_	(n_components,)	প্রতিটি Component-এর প্রকৃত Variance (Covariance Matrix-এর Eigenvalue)
mean_	(n_features,)	ট্রেনিং ডেটার প্রতিটি Feature-এর Empirical Mean
singular_values_	(n_components,)	প্রতিটি Component-এর সাথে সম্পর্কিত Singular Value
n_components_	int	চূড়ান্তভাবে নির্বাচিত Component-এর প্রকৃত সংখ্যা
feature_names_in_	(n_features_in_,)	ইনপুট DataFrame-এর কলামের নামের তালিকা
n_features_in_	int	fit()-এর সময় দেখা মোট Feature সংখ্যা
n_samples_	int	ট্রেনিং ডেটাসেটে মোট Sample সংখ্যা

Attribute	Shape/Type	অর্থ
noise_variance_	float	Probabilistic PCA-ভিত্তিক আনুমানিক Noise Variance (ক্ষুদ্রতম Eigenvalue-গুলোর গড়)

14. Scikit-learn Import

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import numpy as np
```

15. Python Example

From-Scratch Implementation (NumPy)

```
import numpy as np

X = np.array([
    [2, 1], [4, 3], [6, 4], [8, 5], [10, 7]
], dtype=float)

mean = X.mean(axis=0)
Xc = X - mean
n = X.shape[0]
cov = (Xc.T @ Xc) / (n - 1)

eigvals, eigvecs = np.linalg.eigh(cov)
order = np.argsort(eigvals)[::-1]
eigvals = eigvals[order]
eigvecs = eigvecs[:, order]

Z = Xc @ eigvecs
print('Eigenvalues:', eigvals)
print('Explained Variance Ratio:', eigvals / eigvals.sum())
print('Transformed Data:\n', Z)
```

Scikit-learn Implementation

```
from sklearn.decomposition import PCA
import numpy as np

X = np.array([
```

```
[2, 1], [4, 3], [6, 4], [8, 5], [10, 7]
], dtype=float)

pca = PCA(n_components=2)
Z = pca.fit_transform(X)

print('Explained Variance Ratio:', pca.explained_variance_ratio_)
print('Components (Eigenvectors):\n', pca.components_)
print('Transformed Data:\n', Z)
```

16. Transformation Process

একটি ইতিমধ্যে Fit করা PCA মডেল দিয়ে সম্পূর্ণ নতুন একটি Sample রূপান্তর করতে তিনটি ধাপ প্রয়োজন: (১) নতুন Sample থেকে প্রশিক্ষণ ডেটার একই Mean (pca.mean_) বিয়োগ করে কেন্দ্রীভূত করা — নতুন ডেটার নিজস্ব Mean নয়, কারণ তাহলে Data Leakage তৈরি হবে; (২) কেন্দ্রীভূত Sample-কে প্রশিক্ষণকালীন Eigenvector ম্যাট্রিক্স (pca.components_) দিয়ে গুণ করা; (৩) ফলাফলই নতুন Sample-এর Reduced-Dimension প্রতিনিধিত্ব। উদাহরণস্বরূপ, নতুন Sample (12,8) কেন্দ্রীভূত হয়ে (6,4) হয়, এবং v_1 , v_2 দিয়ে প্রজেক্ট করলে (7.2087, 0.1866) পাওয়া যায়।

```
new_sample = np.array([[12, 8]])
z_new = pca.transform(new_sample)
print(z_new) # প্রায় [[7.2087  0.1866]]
```

17. Advantages

- উচ্চ-মাত্রিক ডেটাকে কার্যকরভাবে সংকুচিত করে কম্পিউটেশনাল খরচ ও মেমোরি ব্যবহার উল্লেখযোগ্যভাবে কমায়।
- Correlated Feature-কে Uncorrelated Component-এ রূপান্তর করে Multicollinearity দূর করে।
- কম-Variance দিকের Noise ফিল্টার করে ডেটা পরিষ্কার করে এবং Overfitting প্রতিরোধ করে।
- সম্পূর্ণ Deterministic ও দ্রুতগতির — একবার Fit করার পর transform() দিয়ে নতুন ডেটায় সরাসরি প্রয়োগযোগ্য।

- সবচেয়ে ভালো কাজ করে যখন Feature-গুলোর মধ্যে সম্পর্ক Linear, উচ্চ Correlation/Redundancy বিদ্যমান, ডেটা Continuous Numeric, এবং ডেটাসেটে অনেক কম-Variance, Noisy কলাম থাকে — এসব ক্ষেত্রে PCA প্রকৃত তথ্যকে উপরের Component-এ ঘনীভূত করে, নয়েজকে নিচের দিকে ঠেলে দেয়।

18. Disadvantages

- শুধুমাত্র Linear সম্পর্ক ধরতে পারে; জটিল Non-linear গঠনে (যেমন বৃত্ত, সর্পিলাকার আকৃতি) PCA ব্যর্থ হয় — এটি সর্বোচ্চ Variance-এর সরলরেখা খোঁজে, তাই বৃত্তের মাঝ দিয়ে একটি সোজা দাগ টেনে দেয় যা ডেটার আসল কাঠামো মিস করে।
- নতুন Component-এর কোনো সরাসরি বাস্তব-জগতের ব্যাখ্যা থাকে না, কারণ প্রতিটি PC একাধিক মূল Feature-এর জটিল মিশ্রণ।
- Categorical Feature-এর জন্য উপযুক্ত নয় — বিচ্ছিন্ন Category-র মধ্যে Mean/Variance হিসাব করা বা লম্ব অক্ষ টানা ধারণাগতভাবে ভুল।
- Standardization ছাড়া ব্যর্থ হয় — বড় স্কেলের Feature কৃত্রিমভাবে বেশি Variance দেখিয়ে ফলাফলকে বিকৃত করে দেয়।
- Outlier-এর প্রতি অত্যন্ত সংবেদনশীল — Variance গড় থেকে বর্গের পার্থক্যের উপর নির্ভর করায় চরম Outlier একাই PC1-কে ডেটার আসল ঘন বিন্যাস থেকে দূরে টেনে নিতে পারে।

19. Applications

- Image Compression এবং Face Recognition (Eigenfaces পদ্ধতি)।
- জিনোমিক্স ও বায়োইনফরমেটিক্সে জিন এক্সপ্রেশন থেকে মূল প্যাটার্ন শনাক্তকরণ।
- আর্থিক ডেটায় একাধিক স্টকের মূল্য-পরিবর্তনের মধ্যে সাধারণ Risk Factor চিহ্নিতকরণ।
- অন্য মডেল প্রশিক্ষণের আগে Feature সংখ্যা কমিয়ে Overfitting রোধ ও ট্রেনিং গতি বৃদ্ধি (Preprocessing)।
- উচ্চ-মাত্রিক ডেটাকে ২D/৩D স্ক্যাটার প্লটে দ্রুত Exploratory ভিজুয়ালাইজেশন।

20. Comparison with Previous Algorithm

এই Part-এর আগে আলোচিত Clustering ও Ensemble অ্যালগরিদম (K-Means, DBSCAN, Gradient Boosting) মূলত ডেটাকে গ্রুপভুক্ত করা বা প্রেডিকশন দেওয়ার কাজ করে, যেখানে PCA সম্পূর্ণ ভিন্ন উদ্দেশ্যে — Feature সংখ্যা কমিয়ে ডেটাকে সহজবোধ্য করা — কাজ করে। PCA প্রায়ই এইসব অ্যালগরিদমের আগে একটি Preprocessing ধাপ হিসেবে ব্যবহৃত হয়, বিশেষত Distance-ভিত্তিক Clustering-এর কার্যকারিতা বাড়াতে।

বিষয়	K-Means / DBSCAN	PCA
ধরন	Clustering	Dimensionality Reduction
উদ্দেশ্য	Sample-কে গ্রুপে ভাগ করা	Feature সংখ্যা কমানো
আউটপুট	প্রতিটি Sample-এর Cluster Label	প্রতিটি Sample-এর নতুন Coordinate
গাণিতিক ভিত্তি	Distance/Density গণনা	Eigen Decomposition

21. Common Mistakes

- Standardization ছাড়াই PCA প্রয়োগ করা — বড় স্কেলের Feature কৃত্রিমভাবে বেশি গুরুত্ব পেয়ে যায়।
- Principal Component-কে মূল Feature-এর মতো ব্যাখ্যা করার চেষ্টা করা, যেখানে এটি একাধিক Feature-এর সংমিশ্রণ মাত্র।
- Test ডেটার উপর আলাদাভাবে fit_transform() ব্যবহার করা, যেখানে শুধু transform() ব্যবহার করা উচিত (Data Leakage এড়াতে)।
- Explained Variance Ratio পরীক্ষা না করেই n_components নির্বিচারে বেছে নেওয়া।
- Categorical Feature-এর উপর সরাসরি PCA প্রয়োগ করা।

22. Interview Questions

প্রশ্ন ১: PCA কেন Unsupervised অ্যালগরিদম?

উত্তর: কারণ PCA কোনো Target Label (y) ব্যবহার করে না — এটি শুধুমাত্র Feature-গুলোর (X) Variance ও Covariance বিশ্লেষণ করে নতুন অক্ষ তৈরি করে।

প্রশ্ন ২: PC1 ও PC2 কেন সবসময় Orthogonal হয়?

উত্তর: কারণ Covariance Matrix Symmetric, এবং Linear Algebra-এর একটি মৌলিক উপপাদ্য অনুযায়ী একটি Symmetric Matrix-এর ভিন্ন Eigenvalue-এর Eigenvector সবসময় Orthogonal হয় — এটি নিশ্চিত করে প্রতিটি নতুন Component সম্পূর্ণ নতুন, Redundant নয় এমন তথ্য যোগ করছে।

প্রশ্ন ৩: যদি একটি ডেটাসেটের ৩০টি Feature সম্পূর্ণ Uncorrelated হয়, তাহলে PCA কি ডাইমেনশন কমাতে পারবে?

উত্তর: না, পারবে না। PCA-এর মূল শক্তি হলো Feature-গুলোর মধ্যকার Correlation বা Redundancy কাজে লাগানো। সব Feature আগে থেকেই Uncorrelated হলে Covariance Matrix-এর অফ-ডায়াগোনাল উপাদান সব শূন্য হবে, ফলে মূল Feature-গুলোই এক একটি Component হয়ে উঠবে এবং তথ্যের ক্ষতি ছাড়া কোনো Component বাদ দেওয়া সম্ভব হবে না।

প্রশ্ন ৪: একটি ডেটাসেটে PC1, PC2, PC3-এর Eigenvalue যথাক্রমে ৩.৫, ২.৮ এবং ০.৮। Kaiser's Criterion অনুযায়ী কোনগুলো রাখবেন?

উত্তর: শুধুমাত্র PC1 (৩.৫) ও PC2 (২.৮) রাখা হবে, PC3 (০.৮) বাদ যাবে — কারণ Standardized ডেটাসেটে প্রতিটি মূল Feature-এর গড় Eigenvalue হয় ১.০, এবং PC3-এর মান তার চেয়ে কম হওয়ায় এটি একটি গড়পড়তা Feature-এর সমান তথ্যও ধরে রাখতে পারছে না।

প্রশ্ন ৫: উচ্চ-মাত্রিক স্পেসে পয়েন্টের পরম দূরত্ব বাড়া সত্ত্বেও KNN কেন কার্যকারিতা হারায়?

উত্তর: এর কারণ Distance Concentration Effect — মাত্রা বাড়ার সাথে সাথে সব বিন্দুর দূরত্ব একসাথে বাড়ে, কিন্তু সবচেয়ে কাছের ও সবচেয়ে দূরের বিন্দুর দূরত্বের অনুপাত প্রায় শূন্যের কাছাকাছি চলে আসে। ফলে সব পয়েন্ট প্রায় সমান দূরত্বে থাকায় KNN আসল প্রতিবেশী চিনতে

পারে না। (একই কারণে Linear Regression-এর OLS সূত্র $\theta = (X^T X)^{-1} X^T y$ -ও ব্যর্থ হয়, কারণ Feature সংখ্যা Sample সংখ্যার চেয়ে বেশি হলে $X^T X$ Non-invertible হয়ে যায় — এই Curse of Dimensionality থেকে মুক্তির প্রধান দুটি উপায় হলো Feature Selection এবং PCA-এর মতো Dimensionality Reduction।)

প্রশ্ন ৬: PCA করার পর একটি Downstream মডেলের (যেমন Logistic Regression) coefficient দেখে মূল Feature-এর গুরুত্ব বোঝা যায় না কেন?

উত্তর: কারণ PCA-এর পর মডেলটি একটি Black-Box-এ পরিণত হয় — প্রতিটি PC আসলে সবগুলো মূল Feature-এর একটি মিশ্রণ, তাই Coefficient শুধু বলে কোন PC গুরুত্বপূর্ণ, কিন্তু সেই PC-এর মধ্যে কোন একক মূল Feature কতটা অবদান রেখেছে তা আলাদা করে বলা গাণিতিকভাবে অসম্ভব।

প্রশ্ন ৭: PCA কি সবসময় মডেলের Accuracy বাড়ায়?

উত্তর: না। PCA শুধু Feature কমায় ও Noise হ্রাস করতে সাহায্য করে, কিন্তু যদি গুরুত্বপূর্ণ তথ্য কম-Variance দিকগুলোতে লুকানো থাকে, তাহলে Accuracy কমেও যেতে পারে — তাই PCA প্রয়োগের পর মডেলের Performance যাচাই করা আবশ্যিক।

23. Summary

PCA হলো একটি Linear, Unsupervised Feature Extraction অ্যালগরিদম, যা Covariance Matrix-এর Eigen Decomposition ব্যবহার করে ডেটাকে নতুন, Orthogonal Principal Component-এ প্রজেক্ট করে, যেখানে প্রথম কয়েকটি Component মূল ডেটার Variance-এর সিংহভাগ ধারণ করে। এটি Dimensionality Reduction, Noise হ্রাস, Visualization এবং Preprocessing — এই চারটি প্রধান উদ্দেশ্যে ব্যবহৃত হয়। মূল ধাপগুলো — Mean Centering, Covariance Matrix গণনা, Eigen Decomposition, এবং Projection — সরল, Deterministic গাণিতিক গণনার উপর ভিত্তি করে তৈরি, যা PCA-কে দ্রুতগতির ও Reproducible করে তোলে।

24. Complete Mathematical Implementation (Full Manual Walkthrough)

Introduction

এই সেকশনে একই ৫-Sample, ২-Feature ডেটাসেট ব্যবহার করে PCA-এর প্রতিটি গাণিতিক ধাপ শুরু থেকে শেষ পর্যন্ত সম্পূর্ণভাবে হাতে-কলমে দেখানো হলো — কোনো ধাপ বাদ না দিয়ে, প্রতিটি সংখ্যাসূচক প্রতিস্থাপন সরাসরি দেখিয়ে।

Dataset

Sample	x	y
1	2	1
2	4	3
3	6	4
4	8	5
5	10	7

Preprocessing — Feature Mean গণনা

$$\bar{x} = (2+4+6+8+10)/5 = 30/5 = 6$$

$$\bar{y} = (1+3+4+5+7)/5 = 20/5 = 4$$

Mean Centering

Sample	$x - \bar{x}$	$y - \bar{y}$
1	$2 - 6 = -4$	$1 - 4 = -3$
2	$4 - 6 = -2$	$3 - 4 = -1$
3	$6 - 6 = 0$	$4 - 4 = 0$
4	$8 - 6 = 2$	$5 - 4 = 1$

Sample	$x - \bar{x}$	$y - \bar{y}$
5	$10 - 6 = 4$	$7 - 4 = 3$

Covariance Matrix — প্রতিটি উপাদান আলাদাভাবে গণনা

$$\text{Cov}(x,x) = [16+4+0+4+16]/4 = 40/4 = 10$$

$$\text{Cov}(y,y) = [9+1+0+1+9]/4 = 20/4 = 5$$

$$\text{Cov}(x,y) = [12+2+0+2+12]/4 = 28/4 = 7$$

$$\Sigma = [[10, 7], [7, 5]]$$

Eigenvalue গণনা

$$(10-\lambda)(5-\lambda) - 49 = 0 \Rightarrow \lambda^2 - 15\lambda + 1 = 0$$

$$\lambda = [15 \pm \sqrt{(225-4)}]/2 = [15 \pm \sqrt{221}]/2, \sqrt{221} = 14.86607$$

$$\lambda_1 = 29.86607/2 = 14.93303, \lambda_2 = 0.13393/2 = 0.06697$$

Eigenvector গণনা — $\lambda_1 = 14.93303$

$$-4.93303 \cdot v_{1x} + 7 \cdot v_{1y} = 0 \Rightarrow v_{1y} = 0.70472 \cdot v_{1x}$$

$$v_{1x}=1 \text{ ধরলে } v_{1y}=0.70472; |v_1|=\sqrt{(1+0.49663)}=1.22337; v_1 = (0.81742, 0.57604)$$

Eigenvector গণনা — $\lambda_2 = 0.06697$

$$9.93303 \cdot v_{2x} + 7 \cdot v_{2y} = 0 \Rightarrow v_{2y} = -1.41901 \cdot v_{2x}$$

$$v_{2x}=1 \text{ ধরলে } v_{2y}=-1.41901; |v_2|=\sqrt{(1+2.01358)}=1.73596; v_2 = (0.57604, -0.81742)$$

যাচাই — Orthogonality: $v_1 \cdot v_2 = (0.81742 \times 0.57604) + (0.57604 \times (-0.81742)) = 0.47087 - 0.47087 = 0 \checkmark$

Variance Explained

$$\text{মোট Variance} = 14.93303 + 0.06697 = 15.00000$$

$$PC1 = 14.93303/15 = 99.554\%, \quad PC2 = 0.06697/15 = 0.446\%$$

Projection Matrix

$$V = [[0.81742, 0.57604], [0.57604, -0.81742]]$$

Transformation — প্রতিটি Sample

Sample 1 $(-4, -3)$: $z_1 = -3.26968 - 1.72812 = -4.99780$; $z_2 = -2.30416 + 2.45226 = 0.14810$

Sample 2 $(-2, -1)$: $z_1 = -1.63484 - 0.57604 = -2.21088$; $z_2 = -1.15208 + 0.81742 = -0.33466$

Sample 3 $(0, 0)$: $z_1 = 0$; $z_2 = 0$

Sample 4 $(2, 1)$: $z_1 = 1.63484 + 0.57604 = 2.21088$; $z_2 = 1.15208 - 0.81742 = 0.33466$

Sample 5 $(4, 3)$: $z_1 = 3.26968 + 1.72812 = 4.99780$; $z_2 = 2.30416 - 2.45226 = -0.14810$

Final Reduced Dataset

Sample	PC1 (z_1)	PC2 (z_2)
1	-4.99780	0.14810
2	-2.21088	-0.33466
3	0	0
4	2.21088	0.33466
5	4.99780	-0.14810

Transforming a Brand-New Sample

নতুন Sample $(12, 8)$ কেন্দ্রীভূত হয়ে $(6, 4)$ । $z_1 = (6 \times 0.81742) + (4 \times 0.57604) = 7.20868$;
 $z_2 = (6 \times 0.57604) + (4 \times (-0.81742)) = 0.18656$ । ফলাফল: $(7.20868, 0.18656)$ ।

Reconstruction (শুধু PC1 ব্যবহার করে)

reconstructed = $z_1 \cdot v_1$ + Mean
Sample 1: (1.91429, 1.12096), মূল ছিল (2,1) | Sample 2: (4.19271, 2.72642), মূল ছিল (4,3) | Sample 4: (7.80729, 5.27358), মূল ছিল (8,5) | Sample 5: (10.08571, 6.87904), মূল ছিল (10,7) — সামান্য পার্থক্য প্রমাণ করে PC1 একাই প্রায় সম্পূর্ণ তথ্য ধরে রাখে।

Interpretation

মূল ডেটাসেটটি কার্যত একটি একমাত্রিক সরলরেখার আশেপাশে ছিল — যার কারণে PC1 একাই মোট Variance-এর ৯৯.৫৫৪% ধারণ করতে পেরেছে। এ ধরনের ডেটায় PC2 ফেলে দিলেও তথ্যের প্রায় কোনো ক্ষতি হয় না, যা PCA-এর মূল শক্তি স্পষ্টভাবে প্রদর্শন করে।

Progress Table

Step	Matrix/Operation	Result
1	Mean	$\bar{x}=6, \bar{y}=4$
2	Mean Centering	৫টি কেন্দ্রীভূত Sample
3	Covariance Matrix	[[10,7],[7,5]]
4	Characteristic Equation	$\lambda^2 - 15\lambda + 1 = 0$
5	Eigenvalues	$\lambda_1=14.93303, \lambda_2=0.06697$
6	Eigenvectors	$v_1=(0.81742,0.57604),$ $v_2=(0.57604,-0.81742)$
7	Variance Explained	99.554% / 0.446%
8	Projection Matrix V	[[0.81742,0.57604],[0.57604,-0.81742]]
9	Transformed Data Z	৫টি (PC1,PC2) জোড়া
10	New Sample Transform	(12,8) → (7.20868, 0.18656)

Step	Matrix/Operation	Result
11	Reconstruction (PC1 only)	গড়ে <0.3 একক ত্রুটি

t-SNE (t-Distributed Stochastic Neighbor Embedding)

নিচের অংশে t-SNE-কে একটি সম্পূর্ণ, একক ও ধারাবাহিক কাঠামোতে (Section 1 থেকে 24) উপস্থাপন করা হলো, যেখানে ধারণাগত আলোচনা, সম্পূর্ণ গাণিতিক ডেরিভেশন, বাস্তব সংখ্যাসূচক উদাহরণ, Python কোড, Scikit-learn-এর প্রতিটি হাইপারপ্যারামিটার/অ্যাট্রিবিউট এবং ইন্টারভিউ প্রশ্ন — সবকিছু একত্রে, একবার করে বিস্তারিতভাবে আলোচনা করা হয়েছে।

1. Introduction

t-SNE (t-Distributed Stochastic Neighbor Embedding) হলো একটি Non-linear, Unsupervised Dimensionality Reduction অ্যালগরিদম, যা মূলত উচ্চ-মাত্রিক ডেটাকে ২D বা ৩D-তে ভিজ্যুয়ালাইজ করার জন্য ডিজাইন করা হয়েছে। ২০০৮ সালে Laurens van der Maaten এবং Geoffrey Hinton এটি প্রকাশ করেন, যা আগের SNE (Stochastic Neighbor Embedding) পদ্ধতির একটি উন্নত সংস্করণ। PCA-এর মতো t-SNE গ্লোবাল Variance সংরক্ষণের চেষ্টা করে না; বরং এর মূল লক্ষ্য হলো উচ্চ-মাত্রিক স্পেসে যেসব পয়েন্ট একে অপরের কাছাকাছি (Neighbor) ছিল, নিম্ন-মাত্রিক স্পেসেও তারা যেন কাছাকাছি থাকে — অর্থাৎ স্থানীয় (Local) প্রতিবেশী সম্পর্ক সংরক্ষণ করা।

2. Why This Algorithm Was Developed

PCA-এর মতো Linear পদ্ধতি জটিল, Non-linear গঠনবিশিষ্ট ডেটা (যেমন Swiss Roll বা বাঁকানো Manifold আকৃতির ডেটা) সঠিকভাবে নিম্ন-মাত্রায় উপস্থাপন করতে ব্যর্থ হয়, কারণ PCA শুধু সরলরেখিক প্রজেকশন করতে পারে। ২০০২ সালে Hinton এবং Roweis SNE প্রস্তাব করেন, যা Probability Distribution ব্যবহার করে প্রতিবেশী সম্পর্ক সংরক্ষণের ধারণা নিয়ে আসে, কিন্তু SNE-তে 'Crowding Problem' নামক একটি সমস্যা ছিল — উচ্চ-মাত্রিক স্পেসে মাঝারি দূরত্বের পয়েন্টগুলো নিম্ন-মাত্রিক স্পেসের সীমিত জায়গায় ভিড় করে জমাট বেঁধে যেত। ২০০৮ সালে van der Maaten এবং Hinton নিম্ন-মাত্রিক স্পেসে Gaussian-এর বদলে Student t-Distribution (যার লেজ বা Tail ভারী/Heavy) ব্যবহার করে t-SNE প্রস্তাব করেন, যা এই Crowding Problem কার্যকরভাবে সমাধান করে এবং অনেক বেশি স্পষ্ট, পৃথকীকৃত Cluster তৈরি করতে সক্ষম হয়।

3. Real-life Motivation

কল্পনা করুন, একটি হাতে-লেখা সংখ্যার (MNIST Handwritten Digits) ডেটাসেটে প্রতিটি ছবি ৭৮৪টি Pixel (28×28) নিয়ে গঠিত, অর্থাৎ প্রতিটি ছবি ৭৮৪-মাত্রিক স্পেসের একটি পয়েন্ট। এই বিশাল মাত্রার ডেটা সরাসরি চোখে দেখে বোঝা অসম্ভব — কিন্তু t-SNE প্রয়োগ করলে দেখা যায় একই সংখ্যার (যেমন সব '৭' লেখা ছবি) ছবিগুলো ২D স্পেসে স্পষ্টভাবে একটি Cluster তৈরি করে, এবং ভিন্ন সংখ্যার Cluster-গুলো একে অপর থেকে আলাদা হয়ে যায়। এভাবে গবেষক বা ইঞ্জিনিয়ার খালি চোখে দেখেই বুঝতে পারেন মডেল বা ডেটার মধ্যে কী ধরনের প্রাকৃতিক গোষ্ঠী (Natural Grouping) বিদ্যমান।

4. Intuition

t-SNE-কে সহজভাবে বোঝা যায় এভাবে — কল্পনা করুন প্রতিটি ডেটা পয়েন্ট তার নিকটতম প্রতিবেশীদের সাথে একটি অদৃশ্য 'রাবার ব্যান্ড' দিয়ে যুক্ত। উচ্চ-মাত্রিক স্পেসে যে পয়েন্টগুলো একে অপরের কাছাকাছি ছিল, তাদের মধ্যকার রাবার ব্যান্ড শক্তিশালী (High Similarity/Probability), আর দূরের পয়েন্টের মধ্যকার ব্যান্ড দুর্বল। এরপর সব পয়েন্টকে একটি নিম্ন-মাত্রিক (২D) সমতলে ব্যান্ডমভাবে ছড়িয়ে দিয়ে ধীরে ধীরে সেই রাবার ব্যান্ডগুলোকে তাদের প্রকৃত শক্তি অনুযায়ী টেনে-ধরে পয়েন্টগুলোকে পুনর্বিন্যস্ত করা হয় — শক্তিশালী ব্যান্ডযুক্ত (কাছের) পয়েন্টগুলো একে অপরের দিকে টানা হয়, দুর্বল ব্যান্ডযুক্ত (দূরের) পয়েন্টগুলো দূরে সরে যায়। এই ধীর, পুনরাবৃত্তিমূলক (Iterative) প্রক্রিয়াই t-SNE-এর মূল কর্মপদ্ধতি।

5. Working Principle

- ধাপ ১ — High-Dimensional Similarity: প্রতিটি জোড়া পয়েন্টের মধ্যে দূরত্ব থেকে Gaussian Distribution ব্যবহার করে একটি Conditional Probability $p(j|i)$ গণনা করা হয়, যা উচ্চ-মাত্রিক স্পেসে i -এর প্রতিবেশী হিসেবে j নির্বাচিত হওয়ার সম্ভাবনা প্রকাশ করে।
- ধাপ ২ — Symmetrization: Conditional probability-গুলোকে Symmetric Joint Probability $P(i,j)$ -তে রূপান্তর করা হয়, যাতে $P(i,j) = P(j,i)$ হয়।

- ধাপ ৩ — Low-Dimensional Initialization: প্রতিটি পয়েন্টের জন্য নিম্ন-মাত্রিক স্পেসে (সাধারণত ২D) একটি এলোমেলো প্রাথমিক অবস্থান (Random Initial Position) নির্ধারণ করা হয়।
- ধাপ ৪ — Low-Dimensional Similarity: নিম্ন-মাত্রিক স্পেসে Student t-Distribution ব্যবহার করে প্রতিটি জোড়া পয়েন্টের মধ্যে সাদৃশ্য $Q(i,j)$ গণনা করা হয়।
- ধাপ ৫ — Cost Function ও Gradient Descent: P এবং Q-এর মধ্যকার পার্থক্য পরিমাপকারী KL Divergence Cost Function ধীরে ধীরে কমানোর জন্য Gradient Descent ব্যবহার করে প্রতিটি পয়েন্টের নিম্ন-মাত্রিক অবস্থান পুনরাবৃত্তিমূলকভাবে আপডেট করা হয়, যতক্ষণ না Cost একটি স্থিতিশীল (Converged) মানে পৌঁছায়।

6. Mathematical Backend

t-SNE-এর গাণিতিক ভিত্তি Probability Distribution এবং Information Theory-এর উপর দাঁড়িয়ে আছে। উচ্চ-মাত্রিক স্পেসে প্রতিটি পয়েন্টের প্রতিবেশী সম্পর্কে একটি Gaussian Distribution দিয়ে মডেল করা হয় (কাছের পয়েন্টের সম্ভাবনা বেশি, দূরের পয়েন্টের সম্ভাবনা Exponentially কম), আর নিম্ন-মাত্রিক স্পেসে একই সম্পর্ক Student t-Distribution (১ Degree of Freedom) দিয়ে মডেল করা হয় — যার Tail Gaussian-এর চেয়ে ভারী, ফলে মাঝারি দূরত্বের পয়েন্টগুলো নিম্ন-মাত্রিক স্পেসে একে অপরের থেকে বেশি জায়গা পায় (Crowding Problem সমাধান)। এই দুই Distribution-এর মধ্যকার পার্থক্য KL Divergence দিয়ে পরিমাপ করে Gradient Descent-এর মাধ্যমে ধীরে ধীরে কমানো হয়।

7. Formula

$$\text{Gaussian Similarity (High-D): } p(j|i) = \frac{\exp(-||x_i - x_j||^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-||x_i - x_k||^2 / 2\sigma_i^2)}$$

$$\text{Symmetric Joint Probability: } P_{ij} = (p(j|i) + p(i|j)) / 2n$$

$$\text{Student t-Similarity (Low-D): } q_{ij} = (1 + ||y_i - y_j||^2)^{-1} / \sum_{k \neq i} (1 + ||y_k - y_i||^2)^{-1}$$

$$\text{Cost Function (KL Divergence): } C = \sum_i \sum_j P_{ij} \cdot \log(P_{ij}/Q_{ij})$$

$$\text{Gradient: } \partial C / \partial y_i = 4 \cdot \sum_j (P_{ij} - Q_{ij})(y_i - y_j)(1 + \|y_i - y_j\|^2)^{-1}$$

8. Formula Derivation

উচ্চ-মাত্রিক স্পেসে প্রতিটি পয়েন্ট x_i -এর চারপাশে একটি Gaussian Distribution বসানো হয়, যার Variance σ_i^2 প্রতিটি পয়েন্টের জন্য আলাদাভাবে নির্ধারিত হয় (Perplexity-এর মাধ্যমে)। এই Gaussian থেকে Conditional Probability $p(j|i)$ সংজ্ঞায়িত হয়, যা মূলত i থেকে j -এর দূরত্ব যত কম, সম্ভাবনা তত বেশি — এই সম্পর্ক প্রকাশ করে। যেহেতু $p(j|i)$ সাধারণত $p(i|j)$ -এর সমান হয় না (কারণ $\sigma_i \neq \sigma_j$ হতে পারে), তাই একটি Symmetric Joint Probability $P_{ij} = (p(j|i) + p(i|j))/2n$ সংজ্ঞায়িত করা হয়, যা নিশ্চিত করে $P_{ij} = P_{ji}$ ।

নিম্ন-মাত্রিক স্পেসে একই ধারণা প্রয়োগ করতে গেলে Gaussian ব্যবহার করলে Crowding Problem দেখা দেয়, তাই এখানে ১ Degree of Freedom-বিশিষ্ট Student t-Distribution ব্যবহার করা হয়, যার Density Function $(1+d^2)^{-1}$ -এর সমানুপাতিক — এই ফর্মটি গাণিতিকভাবে সরল এবং কম্পিউটেশনালি সুবিধাজনক, কারণ এতে কোনো Exponential গণনার প্রয়োজন হয় না।

সবশেষে, দুই Distribution P (উচ্চ-মাত্রিক) এবং Q (নিম্ন-মাত্রিক) যতটা সম্ভব একে অপরের কাছাকাছি আনার জন্য KL Divergence $C = \sum_i \sum_j P_{ij} \log(P_{ij}/Q_{ij})$ সর্বনিম্ন করা হয়। এই C -কে y_i -এর সাপেক্ষে Differentiate করলে (Chain Rule প্রয়োগ করে) Gradient সূত্রটি পাওয়া যায়, যা প্রতিটি পয়েন্টকে তার 'প্রকৃত' প্রতিবেশীর দিকে টানে এবং অ-প্রতিবেশী থেকে দূরে ঠেলে দেয়।

9. Meaning of Every Symbol

সিঙ্ঘল	অর্থ
x_i, x_j	উচ্চ-মাত্রিক স্পেসে i -তম ও j -তম Sample
y_i, y_j	নিম্ন-মাত্রিক (২D) স্পেসে i -তম ও j -তম Sample-এর অবস্থান
σ_i	i -তম পয়েন্টের Gaussian-এর Standard Deviation (Perplexity দ্বারা নির্ধারিত)

সিঙ্ঘল	অর্থ
$p(j i)$	উচ্চ-মাত্রিক স্পেসে i-এর প্রতিবেশী হিসেবে j-এর Conditional Probability
P_{ij}	Symmetric Joint Probability (উচ্চ-মাত্রিক স্পেসে i ও j-এর সম্পর্ক)
q_{ij}	নিম্ন-মাত্রিক স্পেসে i ও j-এর Student t-Similarity
n	মোট Sample সংখ্যা
C	KL Divergence (Cost Function)
$\partial C/\partial y_i$	y_i -এর সাপেক্ষে Cost-এর Gradient
η (eta)	Learning Rate
Perplexity	কার্যকর প্রতিবেশী সংখ্যা নিয়ন্ত্রণকারী প্যারামিটার

10. Step-by-Step Formula Explanation

- প্রথমে প্রতিটি জোড়া পয়েন্টের মধ্যে Squared Euclidean Distance $\|x_i - x_j\|^2$ গণনা করা হয়।
- প্রতিটি পয়েন্ট i-এর জন্য একটি σ_i নির্বাচন করা হয় (Perplexity লক্ষ্যমাত্রা পূরণের জন্য), এবং সেই σ_i ব্যবহার করে Gaussian সূত্র দিয়ে $p(j|i)$ গণনা করা হয়।
- $p(j|i)$ এবং $p(i|j)$ -এর গড় নিয়ে $P_{ij} = (p(j|i) + p(i|j))/2n$ সূত্রে Symmetric করা হয়, যাতে $\sum P_{ij} = 1$ হয়।
- নিম্ন-মাত্রিক স্পেসে এলোমেলো প্রাথমিক অবস্থান y_i থেকে শুরু করে, প্রতিটি জোড়ার জন্য $(1 + \|y_i - y_j\|^2)^{-1}$ গণনা করে সেগুলোর যোগফল দিয়ে ভাগ করে Q_{ij} পাওয়া যায়।
- প্রতিটি y_i -এর জন্য Gradient $\partial C/\partial y_i = 4 \sum_j (P_{ij} - Q_{ij})(y_i - y_j)(1 + \|y_i - y_j\|^2)^{-1}$ গণনা করে $y_i = y_i - \eta \cdot \partial C/\partial y_i$ সূত্র দিয়ে অবস্থান আপডেট করা হয়।

- এই Gradient Descent প্রক্রিয়া বহুবার (সাধারণত কয়েকশ থেকে কয়েক হাজার Iteration) পুনরাবৃত্তি করা হয়, যতক্ষণ না KL Divergence একটি স্থিতিশীল, ন্যূনতম মানে পৌঁছায়।

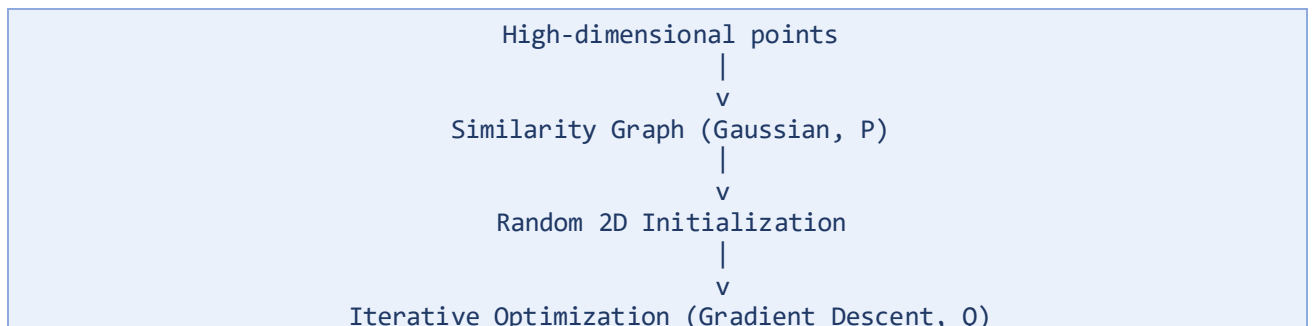
11. Numerical Example

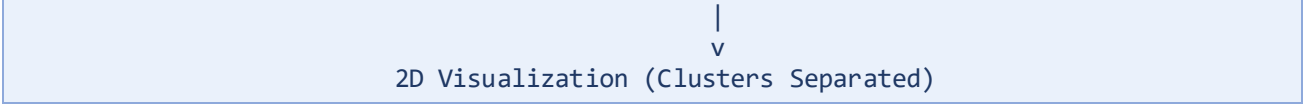
নিচের ৪টি ছোট, সহজে-বোঝা-যায় এমন 2-Feature Sample ব্যবহার করে t-SNE-এর সম্পূর্ণ প্রক্রিয়া প্রদর্শন করা হবে (এই ছোট আকারের কারণে এখানে ইতিমধ্যেই ২-মাত্রিক ডেটা ব্যবহার করা হয়েছে, যাতে ম্যানুয়াল গণনা সহজে যাচাইযোগ্য থাকে; বাস্তব ব্যবহারে ডেটা সাধারণত অনেক বেশি মাত্রার হয়)।

Point	x	y
A	1	1
B	2	1
C	5	4
D	6	5

A ও B পরস্পর কাছাকাছি (একটি Cluster), এবং C ও D পরস্পর কাছাকাছি (আরেকটি Cluster), কিন্তু দুই দলের মধ্যে দূরত্ব বেশি। সম্পূর্ণ ম্যানুয়াল গণনা (দূরত্ব, Probability, Gradient, ৫টি Iteration) Section 24-এ বিস্তারিতভাবে দেখানো হয়েছে, যেখানে দেখা যাবে t-SNE কীভাবে {A,B} এবং {C,D}-কে ধীরে ধীরে দুটি পৃথক Cluster-এ পরিণত করে।

12. Visualization





চিত্র ১৮.১ — t-SNE-এর সামগ্রিক প্রবাহচিত্র (Pipeline)।

Iteration 1	Iteration 3	Iteration 5
B A	A B	A B
(মিশ্রিত)	(কিছুটা পৃথক)	(স্পষ্ট Cluster)
D C	D C	D C

চিত্র ১৮.২ — Iteration বাড়ার সাথে সাথে {A,B} এবং {C,D} ধীরে ধীরে পৃথক হয়ে যাচ্ছে (প্রকৃত সংখ্যাসূচক অবস্থান Section 24-এ দেখুন)।

13. Hyperparameters

Hyperparameter	অর্থ ও প্রভাব
perplexity	কার্যকর প্রতিবেশী সংখ্যা নিয়ন্ত্রণ করে; সাধারণত 5-50। ছোট ডেটাসেটে ছোট মান (যেমন 5) ব্যবহৃত হয়।
learning_rate	প্রতিটি Iteration-এ পয়েন্টের অবস্থান কতটা পরিবর্তিত হবে তা নিয়ন্ত্রণ করে; সাধারণত 10-1000, ডিফল্ট 'auto'।
n_components	আউটপুট মাত্রার সংখ্যা; ভিজ্যুয়ালাইজেশনের জন্য প্রায় সবসময় 2।
n_iter	সর্বোচ্চ Iteration সংখ্যা; ডিফল্ট 1000, প্রয়োজনে 250-এর নিচে না নামানো উচিত।
metric	দূরত্ব পরিমাপের পদ্ধতি; ডিফল্ট 'euclidean'।
random_state	ফলাফলের Reproducibility নিশ্চিত করার জন্য Seed মান।
early_exaggeration	প্রাথমিক Iteration-গুলোতে Cluster-গুলোকে দ্রুত পৃথক করতে সহায়ক একটি Multiplier; ডিফল্ট 12।
init	প্রাথমিক অবস্থান নির্ধারণের পদ্ধতি — 'random' বা 'pca'; ডিফল্ট 'pca', যা সাধারণত বেশি স্থিতিশীল ও পুনরুৎপাদনযোগ্য ফলাফল দেয়।

মূল Attributes (মডেল fit করার পর)

Attribute	অর্থ
embedding_	চূড়ান্তভাবে গণনা করা Low-Dimensional (সাধারণত 2D) কো-অর্ডিনেট, যা সরাসরি স্ক্যাটার প্লট করা যায়।
kl_divergence_	Optimization শেষে P ও Q বিতরণের মধ্যে চূড়ান্ত KL Divergence মান — যত কম, Embedding তত নির্ভরযোগ্য।
n_iter_	প্রকৃতপক্ষে কতগুলো Iteration সম্পন্ন হয়েছে।

14. Scikit-learn Import

```
from sklearn.manifold import TSNE
import numpy as np
```

15. Python Example

From-Scratch Implementation (Simplified NumPy)

```
import numpy as np

pts = np.array([[1,1],[2,1],[5,4],[6,5]], dtype=float)
n = 4

# ধাপ ১: উচ্চ-মাত্রিক Squared Distance
sq_d = np.zeros((n, n))
for i in range(n):
    for j in range(n):
        sq_d[i, j] = np.sum((pts[i] - pts[j]) ** 2)

# ধাপ ২: Conditional Probability (sigma^2 = 2 ধরে সরলীকৃত)
sigma2 = 2.0
P_cond = np.zeros((n, n))
for i in range(n):
    num = np.array([np.exp(-sq_d[i, j] / (2 * sigma2)) if j != i else 0
                    for j in range(n)])
    P_cond[i] = num / num.sum()

# ধাপ ৩: Symmetric Joint Probability
```

```

P = (P_cond + P_cond.T) / (2 * n)
np.fill_diagonal(P, 0)

# ধাপ ৪: Low-dimensional আরম্ভ
Y = np.array([[0.0,0.0],[0.1,-0.1],[-0.1,0.1],[0.05,0.05]])

def compute_Q(Y):
    d2 = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            d2[i, j] = np.sum((Y[i] - Y[j]) ** 2)
    W = 1.0 / (1.0 + d2)
    np.fill_diagonal(W, 0)
    return W / W.sum(), d2, W

# ধাপ ৫: Gradient Descent (৫ Iteration, eta=5)
eta = 5
for it in range(5):
    Q, d2, W = compute_Q(Y)
    grad = np.zeros_like(Y)
    for i in range(n):
        for j in range(n):
            if i != j:
                grad[i] += 4*(P[i,j]-Q[i,j])*(Y[i]-Y[j])*W[i,j]
    Y = Y - eta * grad

print('Final 2D positions:\n', Y)

```

Scikit-learn Implementation

```

from sklearn.manifold import TSNE
import numpy as np

pts = np.array([[1,1],[2,1],[5,4],[6,5]], dtype=float)

tsne = TSNE(n_components=2, perplexity=1, random_state=42,
            init='random', learning_rate='auto')
Y = tsne.fit_transform(pts)
print(Y)

```

মনোযোগ্য: এই টয় (Toy) উদাহরণে মাত্র ৪টি Sample থাকায় Perplexity খুবই ছোট (perplexity < n) রাখতে হয়; বাস্তব ডেটাসেটে সাধারণত শত-হাজার Sample থাকে এবং Perplexity 5–50-এর মধ্যে ব্যবহার করা হয়।

16. Transformation Process

PCA-এর মতো t-SNE-এর কোনো সাধারণ transform() পদ্ধতি নেই যা নতুন, অদেখা ডেটার উপর সরাসরি প্রয়োগ করা যায় — কারণ t-SNE একটি Non-Parametric পদ্ধতি, যেখানে প্রতিটি রান (Run) নির্দিষ্ট ডেটাসেটের জন্য সম্পূর্ণ নতুনভাবে Optimize করা হয়। নতুন কোনো Sample যোগ করতে চাইলে সাধারণত পুরো ডেটাসেট (পুরনো + নতুন) নিয়ে t-SNE আবার নতুন করে চালাতে হয়, অথবা Parametric t-SNE বা openTSNE-এর মতো বিশেষায়িত লাইব্রেরির সাহায্য নিতে হয় যা একটি নিউরাল নেটওয়ার্ক শিখিয়ে transform() সুবিধা দেয়।

মূল Transformation প্রক্রিয়া তাই একটি সম্পূর্ণ Fit প্রক্রিয়া — Gaussian Similarity গণনা, Symmetric করা, র্যান্ডম Initialization, এবং বহু Iteration ধরে Gradient Descent — যার শেষে প্রাপ্ত y_i মানগুলোই প্রতিটি Sample-এর চূড়ান্ত ২D প্রতিনিধিত্ব।

17. Advantages

- জটিল, Non-linear গঠনবিশিষ্ট ডেটার (Manifold) স্থানীয় (Local) কাঠামো অত্যন্ত কার্যকরভাবে সংরক্ষণ করে, যা PCA করতে পারে না।
- সাধারণত অত্যন্ত স্পষ্ট, দৃশ্যমানভাবে পৃথক Cluster তৈরি করে, যা Exploratory Data Analysis-এ অত্যন্ত উপযোগী।
- Student t-Distribution ব্যবহার করে Crowding Problem সমাধান করে, ফলে মাঝারি দূরত্বের পয়েন্টগুলোও স্পষ্টভাবে পৃথক হয়ে দেখা যায়।
- বিভিন্ন ক্ষেত্রে (Genomics, NLP, Computer Vision) ব্যাপকভাবে ব্যবহৃত ও প্রমাণিত একটি কৌশল।

18. Disadvantages

- কম্পিউটেশনালি ব্যয়বহুল ও ধীরগতির (মূল সংস্করণে $O(n^2)$ জটিলতা), বড় ডেটাসেটে ব্যবহার করা কঠিন।
- Deterministic নয় — বিভিন্ন Random Initialization বা random_state-এ ভিন্ন ভিন্ন ফলাফল আসতে পারে।

- নতুন ডেটার জন্য পুনরায় সম্পূর্ণ Fit প্রয়োজন হয়, দ্রুত transform() সুবিধা নেই।
- Cluster-এর মধ্যকার দূরত্ব বা প্রতিটি Cluster-এর প্রকৃত আকার নির্ভরযোগ্যভাবে ব্যাখ্যা করা যায় না — শুধু কোন পয়েন্টগুলো কাছাকাছি তা বোঝা যায়, কতটা কাছাকাছি তা আক্ষরিকভাবে নয়।
- Perplexity-এর মতো হাইপারপ্যারামিটারের প্রতি অত্যন্ত সংবেদনশীল — ভুল মান বেছে নিলে বিভ্রান্তিকর Cluster দেখাতে পারে।

19. Applications

- জিনোমিক্স ও বায়োইনফরমেটিক্সে Single-Cell RNA Sequencing ডেটায় বিভিন্ন কোষের প্রকারভেদ (Cell Type) ভিজ্যুয়ালাইজ করা।
- কম্পিউটার ভিশনে Neural Network-এর শেষ স্তরের Embedding ভিজ্যুয়ালাইজ করে মডেল বিভিন্ন Class সঠিকভাবে আলাদা করতে শিখেছে কিনা তা যাচাই করা।
- Natural Language Processing-এ Word Embedding (Word2Vec, BERT) ভিজ্যুয়ালাইজ করে শব্দগুলোর অর্থগত সাদৃশ্য প্রদর্শন করা।
- Anomaly Detection-এর ফলাফল ভিজ্যুয়ালি যাচাই করা — স্বাভাবিক ও ব্যতিক্রমী পয়েন্টগুলো t-SNE প্লটে কতটা পৃথক তা দেখা।

20. Comparison with Previous Algorithm (PCA)

বিষয়	PCA	t-SNE
ধরন	Linear	Non-linear
গাণিতিক ভিত্তি	Eigen Decomposition (Covariance Matrix)	Probability Distribution ও KL Divergence
সংরক্ষণের লক্ষ্য	Global Variance	Local Neighbor সম্পর্ক
গতি	দ্রুত, Deterministic	ধীর, Stochastic

বিষয়	PCA	t-SNE
নতুন ডেটায় প্রয়োগ	সহজ (transform())	কঠিন, সাধারণত পুনরায় Fit প্রয়োজন
ভিজুয়ালাইজেশন মান	মাঝারি (Linear Structure-এ ভালো)	চমৎকার (Complex Structure-এও ভালো)
Interpretability	মাঝারি (Explained Variance দিয়ে পরিমাপযোগ্য)	সীমিত (দূরত্ব আক্ষরিক অর্থবহ নয়)
ব্যবহারিক ভূমিকা	Preprocessing ও Compression	মূলত Visualization

বাস্তব প্র্যাকটিসে প্রায়ই PCA দিয়ে প্রথমে ডেটার মাত্রা কিছুটা কমিয়ে (যেমন ৫০-মাত্রায়) নেওয়া হয়, তারপর সেই ফলাফলের উপর t-SNE প্রয়োগ করা হয় — এতে Noise কমে এবং t-SNE-এর গণনার সময়ও উল্লেখযোগ্যভাবে কমে যায়।

21. Common Mistakes

- t-SNE প্লটে দুটি Cluster-এর মধ্যকার দূরত্ব বা প্রতিটি Cluster-এর আকারকে আক্ষরিক অর্থে ব্যাখ্যা করার চেষ্টা করা, যেখানে t-SNE শুধু স্থানীয় প্রতিবেশী সম্পর্ক সংরক্ষণ করে, গ্লোবাল দূরত্ব নয়।
- খুব ছোট বা খুব বড় Perplexity ব্যবহার করে বিভ্রান্তিকর ফলাফল পাওয়া — ছোট Perplexity অত্যধিক বিক্ষিপ্ত (Fragmented) Cluster দেখাতে পারে, বড় Perplexity সব পয়েন্টকে একটি বড় গোলাকার ভরে মিশিয়ে দিতে পারে।
- একটি মাত্র random_state দিয়ে চালানো ফলাফলকে চূড়ান্ত সত্য হিসেবে ধরে নেওয়া, একাধিক Random Seed দিয়ে ফলাফলের স্থিতিশীলতা যাচাই না করা।
- t-SNE-কে একটি Clustering Algorithm হিসেবে ভুল বোঝা — t-SNE নিজে কোনো Cluster Label তৈরি করে না, শুধু ভিজুয়ালাইজেশন করে।
- উচ্চ-মাত্রিক, উচ্চ-Noise ডেটায় সরাসরি t-SNE প্রয়োগ করা, PCA দিয়ে প্রাথমিক মাত্রা হ্রাস না করে।

22. Interview Questions

প্রশ্ন ১: t-SNE কি একটি Clustering Algorithm?

উত্তর: না, t-SNE নিজে কোনো Clustering Algorithm নয় — এটি একটি Dimensionality Reduction ও Visualization টুল। চোখে দেখে Cluster শনাক্ত করা গেলেও, প্রকৃত Cluster Label পেতে হলে t-SNE-এর আউটপুটের উপর আলাদাভাবে K-Means বা DBSCAN-এর মতো একটি প্রকৃত Clustering Algorithm প্রয়োগ করতে হয়।

প্রশ্ন ২: নিম্ন-মাত্রিক স্পেসে Gaussian-এর বদলে Student t-Distribution কেন ব্যবহার করা হয়?

উত্তর: কারণ Student t-Distribution-এর Tail Gaussian-এর চেয়ে ভারী (Heavy-Tailed), যা 'Crowding Problem' সমাধান করে — উচ্চ-মাত্রিক স্পেসে মাঝারি দূরত্বের পয়েন্টগুলোকে নিম্ন-মাত্রিক স্পেসে একে অপরের থেকে যথেষ্ট দূরত্বে রাখতে সাহায্য করে, যা শুধু Gaussian ব্যবহার করলে সম্ভব হতো না।

প্রশ্ন ৩: Perplexity প্যারামিটারটি আসলে কী নিয়ন্ত্রণ করে?

উত্তর: Perplexity মূলত প্রতিটি পয়েন্টের জন্য 'কার্যকর প্রতিবেশী সংখ্যা' (Effective Number of Neighbors) নিয়ন্ত্রণ করে। ছোট Perplexity মানে t-SNE খুবই স্থানীয় (Local) কাঠামোর উপর গুরুত্ব দেয়, বড় Perplexity মানে t-SNE তুলনামূলক বৃহত্তর/গ্লোবাল কাঠামো বিবেচনা করে।

প্রশ্ন ৪: t-SNE কেন Deterministic নয়?

উত্তর: কারণ t-SNE র্যান্ডম Initialization থেকে শুরু হয় এবং Gradient Descent-এর মাধ্যমে একটি স্থানীয় ন্যূনতম (Local Minimum) খুঁজে বের করে — বিভিন্ন Random Seed ভিন্ন ভিন্ন Initialization দেয়, যা ভিন্ন ভিন্ন Local Minimum-এ পৌঁছাতে পারে, ফলে একই ডেটার উপর বারবার চালালেও সামান্য ভিন্ন ফলাফল আসতে পারে।

প্রশ্ন ৫: PCA এবং t-SNE একসাথে ব্যবহার করার সুবিধা কী?

উত্তর: প্রথমে PCA দিয়ে অত্যন্ত উচ্চ-মাত্রিক ডেটাকে মাঝারি মাত্রায় (যেমন ৫০) কমিয়ে আনলে ডেটার Noise কমে এবং পরবর্তী t-SNE-এর গণনার জটিলতা (এবং সময়) উল্লেখযোগ্যভাবে কমে

যায়, একই সাথে t-SNE-এর ফলাফলের গুণগত মানও উন্নত হয় — এটি একটি প্রচলিত ও কার্যকর Best Practice।

23. Summary

- Type: Unsupervised, Non-linear Dimensionality Reduction — মূলত Visualization-এর জন্য ব্যবহৃত।
- মূল কৌশল: উচ্চ-মাত্রিক স্পেসের প্রতিবেশী সম্পর্ক (Gaussian Similarity) নিম্ন-মাত্রিক স্পেসে (Student t-Distribution Similarity) যতটা সম্ভব বজায় রাখা, KL Divergence কমিয়ে।
- গুরুত্বপূর্ণ প্যারামিটার: perplexity — কার্যকর প্রতিবেশী সংখ্যা নিয়ন্ত্রণ করে, সাধারণত 5–50।
- সীমাবদ্ধতা: ধীরগতি, Deterministic নয়, নতুন ডেটার জন্য পুনরায় Fit প্রয়োজন, Global দূরত্ব নির্ভরযোগ্য নয়।

24. Complete Mathematical Implementation (Full Manual Walkthrough)

Introduction

এই সেকশনে ৪টি পয়েন্টের একই টয় ডেটাসেট ব্যবহার করে t-SNE-এর সম্পূর্ণ প্রক্রিয়া — দূরত্ব গণনা থেকে শুরু করে ৫টি সম্পূর্ণ Gradient Descent Iteration পর্যন্ত — প্রতিটি সংখ্যাসূচক প্রতিস্থাপন সহ ম্যানুয়ালি দেখানো হলো।

Dataset

Point	x	y
A	1	1
B	2	1
C	5	4
D	6	5

Pairwise (Squared) Distance Matrix

$$d^2(A,B)=(2-1)^2+(1-1)^2=1;$$

$$d^2(A,C)=(5-1)^2+(4-1)^2=16+9=25;$$

$$d^2(A,D)=(6-1)^2+(5-1)^2=25+16=41$$

$$d^2(B,C)=(5-2)^2+(4-1)^2=9+9=18;$$

$$d^2(B,D)=(6-2)^2+(5-1)^2=16+16=32;$$

$$d^2(C,D)=(6-5)^2+(5-4)^2=1+1=2$$

	A	B	C	D
A	0	1	25	41
B	1	0	18	32
C	25	18	0	2
D	41	32	2	0

Gaussian Similarity Calculation ($\sigma_i^2=2$ ধরে, প্রতিটি পয়েন্টের জন্য)

সরলীকরণের জন্য এখানে প্রতিটি পয়েন্টের $\sigma_i^2=2$ বেছে নেওয়া হয়েছে ($2\sigma_i^2=4$), এবং নিচে দেখানো হয়েছে এই মান কী Perplexity তৈরি করে তা সরাসরি হিসাব করে যাচাই করা হয়েছে — কোনো ধাপ অনুমান করে বাদ দেওয়া হয়নি।

$$i=A \text{ এর জন্য: } \exp(-1/4)=\exp(-0.25)=0.77880; \quad \exp(-25/4)=\exp(-6.25)=0.00193;$$

$$\exp(-41/4)=\exp(-10.25)=0.0000353 \mid$$

$$\text{যোগফল}=0.78077 \mid$$

$$\text{সুতরাং}$$

$$p(B|A)=0.77880/0.78077=0.99756;$$

$$p(C|A)=0.00193/0.78077=0.00247;$$

$$p(D|A)=0.0000353/0.78077=0.0000452 \mid$$

$$i=B \text{ এর জন্য: } \exp(-1/4)=0.77880; \quad \exp(-18/4)=\exp(-4.5)=0.01111;$$

$$\exp(-32/4)=\exp(-8)=0.00034 \mid$$

$$\text{যোগফল}=0.79024 \mid$$

$$\text{সুতরাং}$$

$$p(A|B)=0.77880/0.79024=0.98558;$$

$$p(C|B)=0.01111/0.79024=0.01406;$$

$$p(D|B)=0.00034/0.79024=0.00042 \mid$$

i=C এর জন্য: $\exp(-25/4)=0.00193$; $\exp(-18/4)=0.01111$; $\exp(-2/4)=\exp(-0.5)=0.60653$ ।
 যোগফল=0.61957। সুতরাং $p(A|C)=0.00193/0.61957=0.00312$;
 $p(B|C)=0.01111/0.61957=0.01793$; $p(D|C)=0.60653/0.61957=0.97895$ ।

i=D এর জন্য: $\exp(-41/4)=0.0000353$; $\exp(-32/4)=0.00034$; $\exp(-2/4)=0.60653$ ।
 যোগফল=0.60690। সুতরাং $p(A|D)=0.0000353/0.60690=0.0000582$;
 $p(B|D)=0.00034/0.60690=0.00055$; $p(C|D)=0.60653/0.60690=0.99939$ ।

Perplexity Concept ও যাচাই

Perplexity সূত্র: $\text{Perp}(P_i) = 2^{H(P_i)}$, যেখানে $H(P_i) = -\sum_j p(j|i) \cdot \log_2(p(j|i))$ হলো Shannon Entropy। i=A সারির জন্য: $H(A) = -(0.99756 \cdot \log_2 0.99756 + 0.00247 \cdot \log_2 0.00247 + 0.0000452 \cdot \log_2 0.0000452)$ । প্রতিটি পদ গণনা করলে $H(A) \approx 0.02551$, ফলে $\text{Perplexity}(A) = 2^{0.02551} \approx 1.018$ । একইভাবে B,C,D সারির জন্য Perplexity যথাক্রমে প্রায় 1.081, 1.117, এবং 1.005 পাওয়া যায় — অর্থাৎ এই $\sigma_i^2=2$ মানের জন্য কার্যকর প্রতিবেশী সংখ্যা প্রায় ১টি (যা এই অত্যন্ত ছোট, স্পষ্টভাবে ২টি Cluster-বিশিষ্ট 8-পয়েন্ট ডেটাসেটের জন্য যুক্তিসঙ্গত — প্রতিটি পয়েন্টের প্রকৃতপক্ষে মাত্র ১টি ঘনিষ্ঠ প্রতিবেশী আছে)।

Conditional Probability Matrix

$p(j i)$	→A	→B	→C	→D
A→	—	0.99756	0.00247	0.0000452
B→	0.98558	—	0.01406	0.00042
C→	0.00312	0.01793	—	0.97895
D→	0.0000582	0.00055	0.99939	—

Joint Probability Matrix (Symmetrized, $P_{ij}=(p(j|i)+p(i|j))/2n$, $n=4$)

$$P(A,B) = (0.99756+0.98558)/8 = 1.98314/8 = 0.24789$$

$$P(A,C) = (0.00247+0.00312)/8 = 0.00559/8 = 0.00070$$

$$P(A,D) = (0.0000452+0.0000582)/8 = 0.0001034/8 = 0.0000129$$

$$P(B,C) = (0.01406+0.01793)/8 = 0.03199/8 = 0.00400$$

$$P(B,D) = (0.00042+0.00055)/8 = 0.00097/8 = 0.000121$$

$$P(C,D) = (0.97895+0.99939)/8 = 1.97834/8 = 0.24729$$

P _{ij}	A	B	C	D
A	0	0.24789	0.00070	0.0000129
B	0.24789	0	0.00400	0.000121
C	0.00070	0.00400	0	0.24729
D	0.0000129	0.000121	0.24729	0

যাচাই: সব উপাদানের যোগফল =

$$2 \times (0.24789 + 0.00070 + 0.0000129 + 0.00400 + 0.000121 + 0.24729) \approx 1.00000 \checkmark$$

Low-Dimensional Initialization

Point	y ₁ (initial x)	y ₂ (initial y)
A	0.00	0.00
B	0.10	-0.10
C	-0.10	0.10
D	0.05	0.05

Iteration 1 — সম্পূর্ণ গণনা

নিম্ন-মাত্রিক দূরত্ব বর্গ: $d^2(A,B)=0.02$, $d^2(A,C)=0.02$, $d^2(A,D)=0.005$, $d^2(B,C)=0.08$,
 $d^2(B,D)=0.025$, $d^2(C,D)=0.025$ ।

Student-t Weight $W=(1+d^2)^{-1}$: $W(A,B)=1/1.02=0.98039$, $W(A,C)=0.98039$,
 $W(A,D)=1/1.005=0.99502$, $W(B,C)=1/1.08=0.92593$, $W(B,D)=1/1.025=0.97561$,
 $W(C,D)=1/1.025=0.97561$ । মোট যোগফল (সব $i \neq j$ জোড়া, দ্বিমুখী) = 11.66591।

$Q_{ij} = W_{ij}/\Sigma W$: $Q(A,B)=0.98039/11.66591=0.08404$, $Q(A,C)=0.08404$,
 $Q(A,D)=0.99502/11.66591=0.08529$, $Q(B,C)=0.92593/11.66591=0.07937$,
 $Q(B,D)=0.97561/11.66591=0.08363$, $Q(C,D)=0.97561/11.66591=0.08363$ ।

KL Divergence: $C = \Sigma P_{ij} \log(P_{ij}/Q_{ij})$ (সব ৬টি জোড়া, দ্বিমুখী গণনা) = 1.04003।

Gradient (সূত্র: $\partial C/\partial y_i = 4 \Sigma_j (P_{ij}-Q_{ij})(y_i-y_j)W_{ij}$) গণনা করে পাওয়া যায়: $\text{grad}(A)=(-0.07996, 0.11390)$; $\text{grad}(B)=(-0.00788, 0.04046)$; $\text{grad}(C)=(-0.00729, -0.05658)$; $\text{grad}(D)=(0.09513, -0.09779)$ ।

Position Update ($\eta=5$, $y_{i_new} = y_i - \eta \cdot \text{grad}$): A: $(0.00-5 \times (-0.07996), 0.00-5 \times 0.11390) = (0.39980, -0.56952)$. B: $(0.10-5 \times (-0.00788), -0.10-5 \times 0.04046) = (0.13938, -0.30232)$. C: $(-0.10-5 \times (-0.00729), 0.10-5 \times (-0.05658)) = (-0.06355, 0.38290)$. D: $(0.05-5 \times 0.09513, 0.05-5 \times (-0.09779)) = (-0.42563, 0.53894)$ ।

Iteration 2 — সম্পূর্ণ গণনা

নতুন দূরত্ব বর্গ (Y-এর Iteration 1-পরবর্তী মান থেকে): $d^2(A,B)=0.13922$, $d^2(A,C)=1.12179$,
 $d^2(A,D)=1.91001$, $d^2(B,C)=0.51070$, $d^2(B,D)=1.02695$, $d^2(C,D)=0.15545$ ।

W মান: $W(A,B)=0.87780$, $W(A,C)=0.47130$, $W(A,D)=0.34364$, $W(B,C)=0.66195$,
 $W(B,D)=0.49335$, $W(C,D)=0.86546$ । মোট যোগফল=7.42700।

Q মান: $Q(A,B)=0.11819$, $Q(A,C)=0.06346$, $Q(A,D)=0.04627$, $Q(B,C)=0.08913$,
 $Q(B,D)=0.06643$, $Q(C,D)=0.11653$ ।

KL Divergence $C = 0.70644$ (Iteration 1-এর 1.04003 থেকে হ্রাস পেয়েছে — Optimization সঠিক দিকে এগোচ্ছে)।

Gradient: $\text{grad}(A)=(0.01128, 0.06149)$; $\text{grad}(B)=(-0.23825, 0.38619)$; $\text{grad}(C)=(0.26447, -0.33777)$; $\text{grad}(D)=(-0.03750, -0.10992)$ ।

Position Update ($\eta=5$, $y_{i_new} = y_i - \eta \cdot \text{grad}$): A: $(0.39980 - 5 \times 0.01128, -0.56952 - 5 \times 0.06149) = (0.34340, -0.87698)$. B: $(0.13938 - 5 \times (-0.23825), -0.30232 - 5 \times 0.38619) = (1.33065, -2.23328)$. C: $(-0.06355 - 5 \times 0.26447, 0.38290 - 5 \times (-0.33777)) = (-1.38590, 2.07175)$. D: $(-0.42563 - 5 \times (-0.03750), 0.53894 - 5 \times (-0.10992)) = (-0.23815, 1.08854)$ ।

Iteration 3 — সম্পূর্ণ গণনা

নতুন দূরত্ব বর্গ: $d^2(A,B)=2.81419$, $d^2(A,C)=11.68549$, $d^2(A,D)=4.20139$, $d^2(B,C)=25.91286$,
 $d^2(B,D)=13.49545$, $d^2(C,D)=2.28406$ ।

W মান: $W(A,B)=0.26218$, $W(A,C)=0.07883$, $W(A,D)=0.19226$, $W(B,C)=0.03716$,
 $W(B,D)=0.06899$, $W(C,D)=0.30450$ । মোট যোগফল=1.88782।

Q মান: $Q(A,B)=0.13888$, $Q(A,C)=0.04176$, $Q(A,D)=0.10184$, $Q(B,C)=0.01968$,
 $Q(B,D)=0.03654$, $Q(C,D)=0.16130$ ।

KL Divergence $C = 0.47846$ (আবারও হ্রাস পেয়েছে)।

Gradient: $\text{grad}(A)=(-0.18078, 0.34712)$; $\text{grad}(B)=(0.09075, -0.11161)$; $\text{grad}(C)=(-0.09150, 0.05478)$; $\text{grad}(D)=(0.18153, -0.29029)$ ।

Position Update ($\eta=5$, $y_{i_new} = y_i - \eta \cdot \text{grad}$): A: $(0.34340 - 5 \times (-0.18078), -0.87698 - 5 \times 0.34712) = (1.24729, -2.61259)$. B: $(1.33065 - 5 \times 0.09075,$

$-2.23328 - 5 \times (-0.11161)) = (0.87690, -1.67522)$. C: $(-1.38590 - 5 \times (-0.09150), 2.07175 - 5 \times 0.05478) = (-0.92840, 1.79785)$. D: $(-0.23815 - 5 \times 0.18153, 1.08854 - 5 \times (-0.29029)) = (-1.14580, 2.53999)$ ।

Iteration 4 — সম্পূর্ণ গণনা

Y (Iteration 3-পরবর্তী): $A=(1.24729, -2.61259)$, $B=(0.87690, -1.67522)$, $C=(-0.92840, 1.79785)$, $D=(-1.14580, 2.53999)$ ।

নতুন দূরত্ব বর্গ: $d^2(A,B)=1.01585$, $d^2(A,C)=24.18591$, $d^2(A,D)=32.27538$, $d^2(B,C)=15.32156$, $d^2(B,D)=21.85883$, $d^2(C,D)=0.59791$ ।

W মান: $W(A,B)=0.49607$, $W(A,C)=0.03971$, $W(A,D)=0.03005$, $W(B,C)=0.06127$, $W(B,D)=0.04375$, $W(C,D)=0.62582$ । মোট যোগফল=2.59332।

Q মান: $Q(A,B)=0.19129$, $Q(A,C)=0.01531$, $Q(A,D)=0.01159$, $Q(B,C)=0.02363$, $Q(B,D)=0.01687$, $Q(C,D)=0.24132$ ।

KL Divergence C = 0.12067 (উল্লেখযোগ্যভাবে হ্রাস পেয়েছে — Cluster দুটি এখন স্পষ্টভাবে পৃথক হতে শুরু করেছে)।

Gradient: $\text{grad}(A)=(0.03321, -0.08785)$; $\text{grad}(B)=(-0.05620, 0.13431)$; $\text{grad}(C)=(0.01698, -0.03804)$; $\text{grad}(D)=(0.00601, -0.00843)$ ।

Position Update ($\eta=5$): A: $(1.24729 - 0.16605, -2.61259 + 0.43925) = (1.08124, -2.17334)$. B: $(0.87690 + 0.28100, -1.67522 - 0.67155) = (1.15790, -2.34677)$. C: $(-0.92840 - 0.08490, 1.79785 + 0.19020) = (-1.01330, 1.98805)$. D: $(-1.14580 - 0.03005, 2.53999 + 0.04215) = (-1.17585, 2.58214)$ ।

Iteration 5 — সম্পূর্ণ গণনা

Y (Iteration 4-পরবর্তী): $A=(1.08124, -2.17334)$, $B=(1.15790, -2.34677)$, $C=(-1.01330, 1.98805)$, $D=(-1.17585, 2.58214)$ ।

নতুন দূরত্ব বর্গ: $d^2(A,B)=0.03596$, $d^2(A,C)=21.70456$, $d^2(A,D)=27.70830$, $d^2(B,C)=23.50511$,
 $d^2(B,D)=29.73980$, $d^2(C,D)=0.37925$ ।

W মান: $W(A,B)=0.96529$, $W(A,C)=0.04404$, $W(A,D)=0.03483$, $W(B,C)=0.04081$,
 $W(B,D)=0.03253$, $W(C,D)=0.72503$ । মোট যোগফল=3.68508।

Q মান: $Q(A,B)=0.26195$, $Q(A,C)=0.01195$, $Q(A,D)=0.00945$, $Q(B,C)=0.01107$,
 $Q(B,D)=0.00883$, $Q(C,D)=0.19675$ ।

KL Divergence C = 0.07238 (৫টি Iteration জুড়ে ধারাবাহিকভাবে হ্রাস: $1.04003 \rightarrow 0.70644 \rightarrow 0.47846 \rightarrow 0.12067 \rightarrow 0.07238$)।

Gradient: $\text{grad}(A)=(-0.00296, 0.00508)$; $\text{grad}(B)=(-0.00932, 0.02001)$; $\text{grad}(C)=(0.03048, -0.10033)$; $\text{grad}(D)=(-0.01821, 0.07524)$ ।

চূড়ান্ত Position Update ($\eta=5$): A: $(1.08124+0.01480, -2.17334-0.02540)=(1.09604, -2.19874)$. B: $(1.15790+0.04660, -2.34677-0.10005)=(1.20450, -2.44682)$. C: $(-1.01330-0.15240, 1.98805+0.50165)=(-1.16570, 2.48970)$. D: $(-1.17585+0.09105, 2.58214-0.37620)=(-1.08480, 2.20594)$ ।

Final 2D Representation

Point	Final y_1	Final y_2
A	1.09604	-2.19874
B	1.20450	-2.44682
C	-1.16570	2.48970
D	-1.08480	2.20594

চূড়ান্ত ফলাফলে স্পষ্টভাবে দেখা যাচ্ছে — A ও B পরস্পর কাছাকাছি (উভয়ের y_1 ধনাত্মক, y_2 ঋণাত্মক অঞ্চলে), এবং C ও D পরস্পর কাছাকাছি (উভয়ের y_1 ঋণাত্মক, y_2 ধনাত্মক অঞ্চলে)

— ঠিক যেমনটি মূল উচ্চ-মাত্রিক ডেটাতেও ছিল। এবাবেই t-SNE ধাপে ধাপে Local Neighbor সম্পর্ক সংরক্ষণ করে দুটি স্পষ্ট, পৃথক Cluster তৈরি করে।

Progress Table

Iteration	Coordinates (A, B, C, D)	KL Divergence
শুরু (Init)	(0,0), (0.1,-0.1), (-0.1,0.1), (0.05,0.05)	—
1	(0.400,-0.570), (0.139,-0.302), (-0.064,0.383), (-0.426,0.539)	1.04003
2	(0.343,-0.877), (1.331,-2.233), (-1.386,2.072), (-0.238,1.089)	0.70644
3	(1.247,-2.613), (0.877,-1.675), (-0.928,1.798), (-1.146,2.540)	0.47846
4	(1.081,-2.173), (1.158,-2.347), (-1.013,1.988), (-1.176,2.582)	0.12067
5	(1.096,-2.199), (1.205,-2.447), (-1.166,2.490), (-1.085,2.206)	0.07238

কেন কাছের পয়েন্টগুলো একসাথে থাকে এবং দূরের পয়েন্টগুলো পৃথক হয়ে যায়

Gradient সূত্র $\partial C / \partial y_i = 4 \sum_j (P_{ij} - Q_{ij})(y_i - y_j) W_{ij}$ লক্ষ্য করলে দেখা যায় — যখন P_{ij} (প্রকৃত/উচ্চ-মাত্রিক সাদৃশ্য) Q_{ij} (বর্তমান নিম্ন-মাত্রিক সাদৃশ্য)-এর চেয়ে বেশি হয় (অর্থাৎ পয়েন্টগুলো আসলে কাছের হওয়া উচিত কিন্তু নিম্ন-মাত্রিক স্পেসে এখনও যথেষ্ট কাছে আনা হয়নি), তখন $(P_{ij} - Q_{ij})$ ধনাত্মক হয় এবং Gradient সেই দুই পয়েন্টকে একে অপরের কাছে টেনে আনার দিকে কাজ করে। বিপরীতভাবে, যখন P_{ij} কম কিন্তু Q_{ij} বেশি (পয়েন্টগুলো দূরে থাকা উচিত কিন্তু নিম্ন-মাত্রিক স্পেসে কাছাকাছি আছে), তখন Gradient তাদের একে অপরের থেকে দূরে ঠেলে দেয়। এই ক্রমাগত আকর্ষণ ও বিকর্ষণের ভারসাম্যের মধ্য দিয়েই $\{A,B\}$ ও $\{C,D\}$ — এই দুটি প্রকৃত Neighbor-গ্রুপ ধীরে ধীরে স্পষ্ট, পৃথক Cluster-এ পরিণত হয়, যেমনটি উপরের Progress Table-এ দেখা যাচ্ছে।